SYDE 671 · ASSIGNMENT 2 · FALL 2026

Spatial Filtering and Multiresolution Blending

Moksh Trehan · October 19, 2026

Experiments in convolution, image gradients, sharpening, hybrid images, and multiresolution blending. Each section presents the method, visual results, and observations.

Contents

1. **Part One · Filters**
    1. Convolutions from scratch
    2. Finite differences and thresholding
    3. Derivatives of Gaussians
    4. Gradient orientations
2. **Part Two · Frequencies**
    1. Image sharpening
    2. Hybrid images
    3. Gaussian and Laplacian stacks
    4. Multiresolution blending

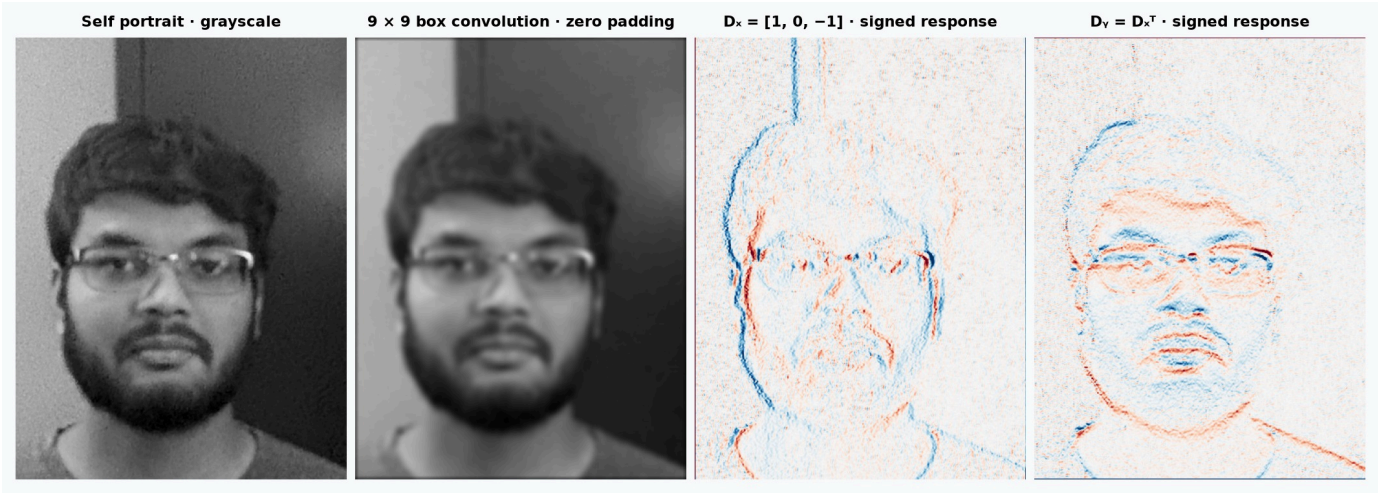
Implementation appendix · Reproduction · Image credits

Online report: https://mokshtrehan.github.io/syde671-assignment2/

1.1 Convolutions from scratch

Convolution flips a kernel in both dimensions, then sums the products of overlapping samples. I implemented this operation using four loops and two loops, and applied a box filter and two derivative filters to the self portrait.

Filters $B = (1/81) \cdot \mathbf{1}_{9\times9}$ $D_x = [1, 0, -1]$ $D_y = D_x^T$



Self portrait, box-filtered image, and signed horizontal and vertical derivatives. Red indicates positive responses and blue indicates negative responses. The shared derivative scale clips the strongest responses for display.

Filtering the portrait

The box filter removes fine texture. The horizontal and vertical derivatives emphasize intensity changes across columns and rows. Their signed responses share a symmetric color scale. Zero padding darkens the outer rim of the smoothed image and can introduce derivative responses at the frame.

The working image is 384 × 512 pixels. Grayscale intensity is 0.299R + 0.587G + 0.114B, with RGB values in [0, 1].

Implementation

The four-loop version iterates over image rows, image columns, kernel rows, and kernel columns. The two-loop version retains the kernel loops and multiplies complete shifted image slices. Both use NumPy operations only; neither calls a convolution, FFT, window-view, or image-processing routine.

Appendices A and B contain the input checks, padding, and both complete implementations.

1.1 Runtime and boundary effects

Maximum absolute error is the largest pixelwise difference from SciPy; runtime is the median elapsed time per call. Both implementations agree to floating-point precision. Asymmetric odd and even kernels test reversal and alignment, which a symmetric Gaussian alone cannot reveal.

Runtime and numerical agreement · 96 × 96 float64 input

Kernel	Method	Median time (ms)	Max. absolute error
3×5 asymmetric	Four loops	43.908	5.55e-16
3×5 asymmetric	Two loops	0.123	5.55e-16
3×5 asymmetric	SciPy convolve2d	0.209	0.00e+00
2×4 asymmetric (even)	Four loops	23.082	8.88e-16
2×4 asymmetric (even)	Two loops	0.08	8.88e-16
2×4 asymmetric (even)	SciPy convolve2d	0.138	0.00e+00
9×9 box	Four loops	209.111	7.77e-16
9×9 box	Two loops	0.609	7.77e-16
9×9 box	SciPy convolve2d	0.9	0.00e+00
1×3 derivative	Four loops	9.022	0.00e+00
1×3 derivative	Two loops	0.047	0.00e+00
1×3 derivative	SciPy convolve2d	0.095	0.00e+00

Both manual methods require O(HWKhKw) arithmetic. The two-loop version removes Python's per-pixel loop overhead by operating on image slices. These timings are local measurements, using a 96 × 96 float64 image and the median of three calls after one warm-up.

What happens at the image boundary?

Boundary values for a constant image

SciPy boundary setting	Corner	Top edge midpoint	Center
fill	0.308642	0.555556	1.0
symm	1.0	1.0	1.0
wrap	1.0	1.0	1.0

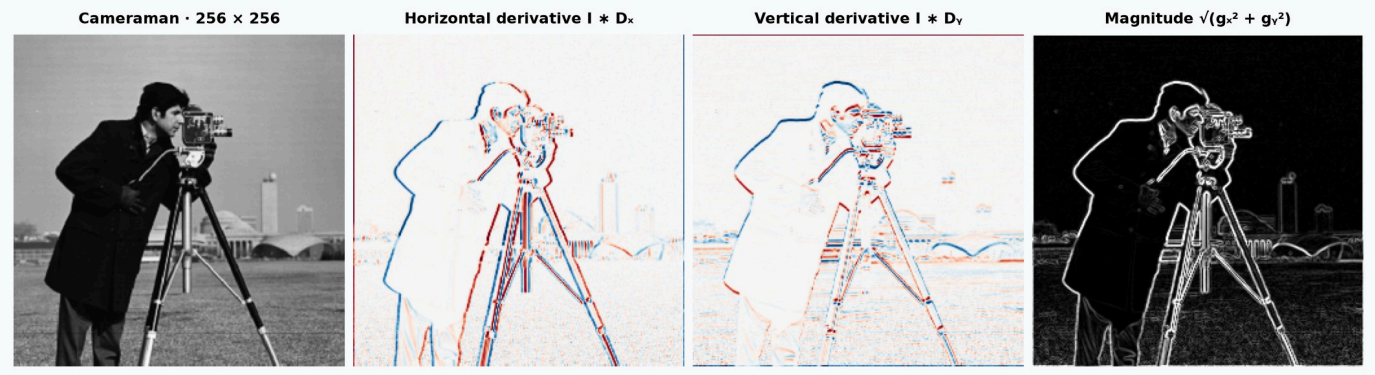
For a constant 25 × 25 image and a normalized 9 × 9 box filter, zero padding gives 25/81 at a corner and 5/9 at an edge midpoint. Symmetric and periodic extension preserve one. The "same" option sets output size; the boundary option supplies samples outside the image.

Along each axis, the manual implementations pad by floor(K/2) before the image and floor((K–1)/2) after it. This matches SciPy's "same" convention for even kernels. The largest comparison error is 8.88×10^{-16}.

1.2 Finite difference operator

The cameraman image is differentiated horizontally and vertically. Gradient magnitude combines these signed responses into an unsigned measure of edge strength.

Gradient $g_x = I * D_x \quad g_y = I * D_y \quad |\nabla I| = \sqrt{(g_x^2 + g_y^2)}$



Cameraman finite differences and gradient magnitude. Signed derivatives share the range [−0.65, 0.65]; magnitude uses [0, 0.65].

Interpreting the derivatives

The horizontal response emphasizes intensity changes across columns; the vertical response emphasizes changes across rows. Convolution reverses the derivative kernel, so an increase from left to right produces a positive horizontal response.

The white border is cropped from the supplied cameraman image before resizing it to 256 × 256 pixels. Computation uses floating-point intensities in [0, 1]. The complete crop coordinates and preprocessing steps are recorded in Appendix B.

1.2 Choosing an edge threshold

A pixel is classified as an edge when its gradient magnitude exceeds T. Three thresholds show the balance between retaining weak contours and rejecting background texture.



Threshold comparison for the finite-difference magnitude. The selected threshold is 0.16. The black outer frame is excluded from evaluation.

Threshold sensitivity

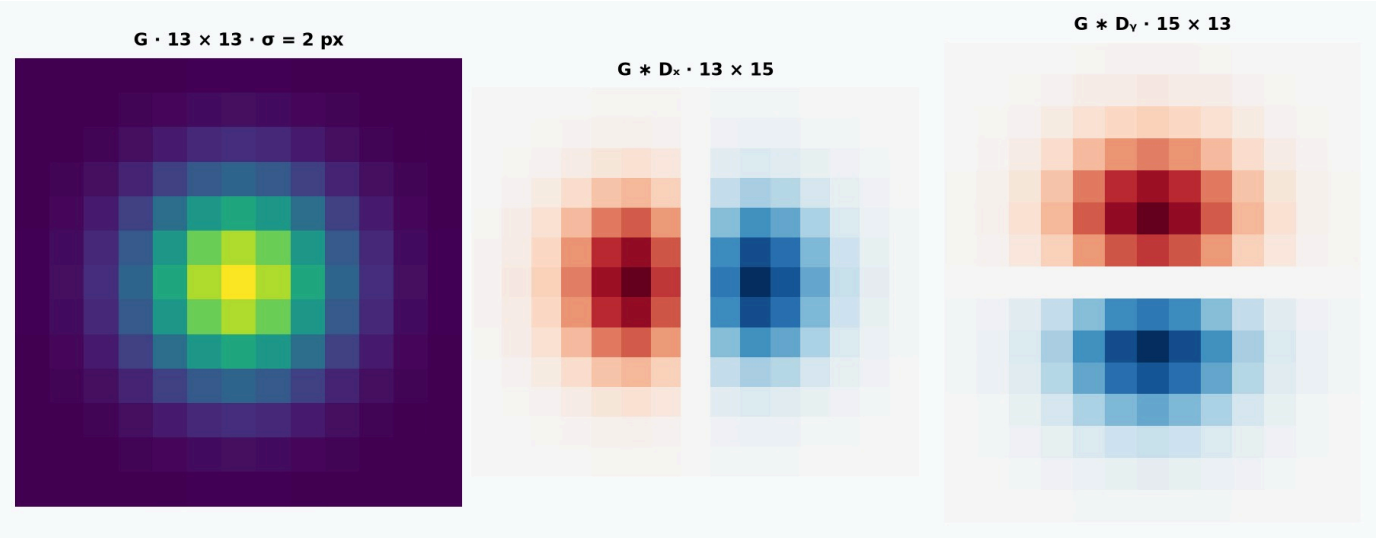
Threshold	Interior edge fraction	Selection
0.08	23.68%	Comparison
0.16	12.81%	Selected
0.24	8.99%	Comparison

A threshold of 0.16 preserves the person, camera, and tripod while rejecting much of the ground texture visible at 0.08. Raising it to 0.24 removes more weak contours. This selection is based on the images; no annotated edge map is used to measure detection accuracy.

A seven-pixel frame is excluded from all Part 1 binary maps and edge-fraction measurements. This removes responses caused by zero padding and covers the largest Gaussian-derivative support. The signed derivative and magnitude images show the complete responses.

1.3 Derivative of Gaussian

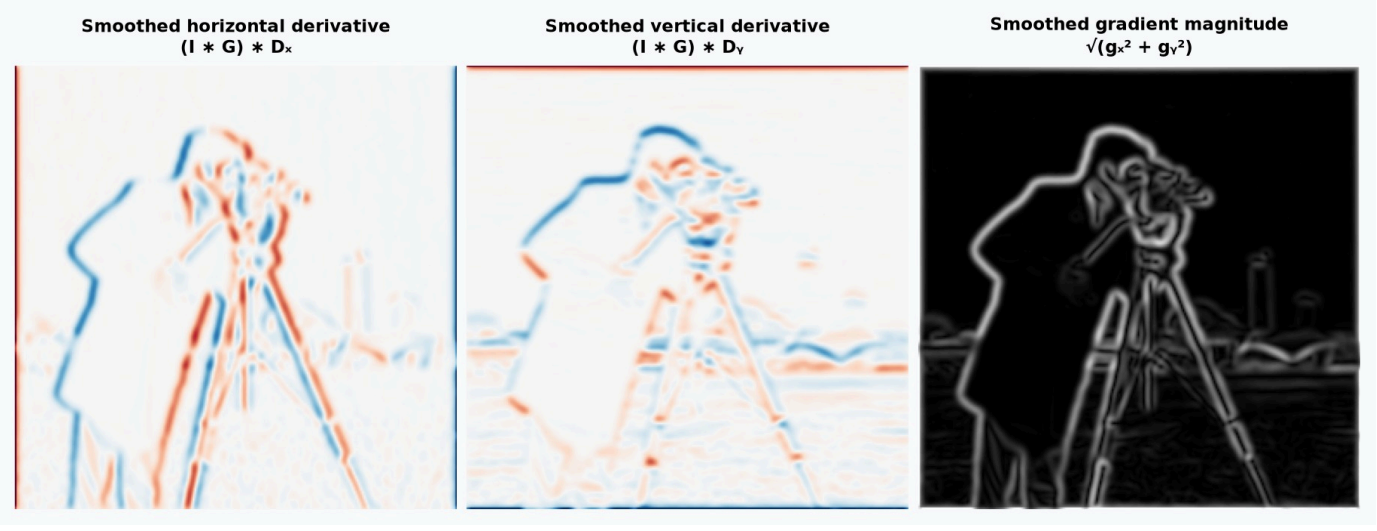
Smoothing before differentiation reduces the response to fine texture. A 13 × 13 Gaussian with σ = 2 pixels is formed from the outer product of cv2.getGaussianKernel(13, 2) with itself.



Gaussian kernel and its sampled horizontal and vertical derivatives. The derivative kernels share a signed scale; the Gaussian uses a separate nonnegative scale.

Convolving this Gaussian with the finite-difference kernels gives a single filter for each derivative. Full kernel convolution produces supports of 13 × 15 and 15 × 13 pixels.

Combined filters $K_x = G * D_x \quad K_y = G * D_y$



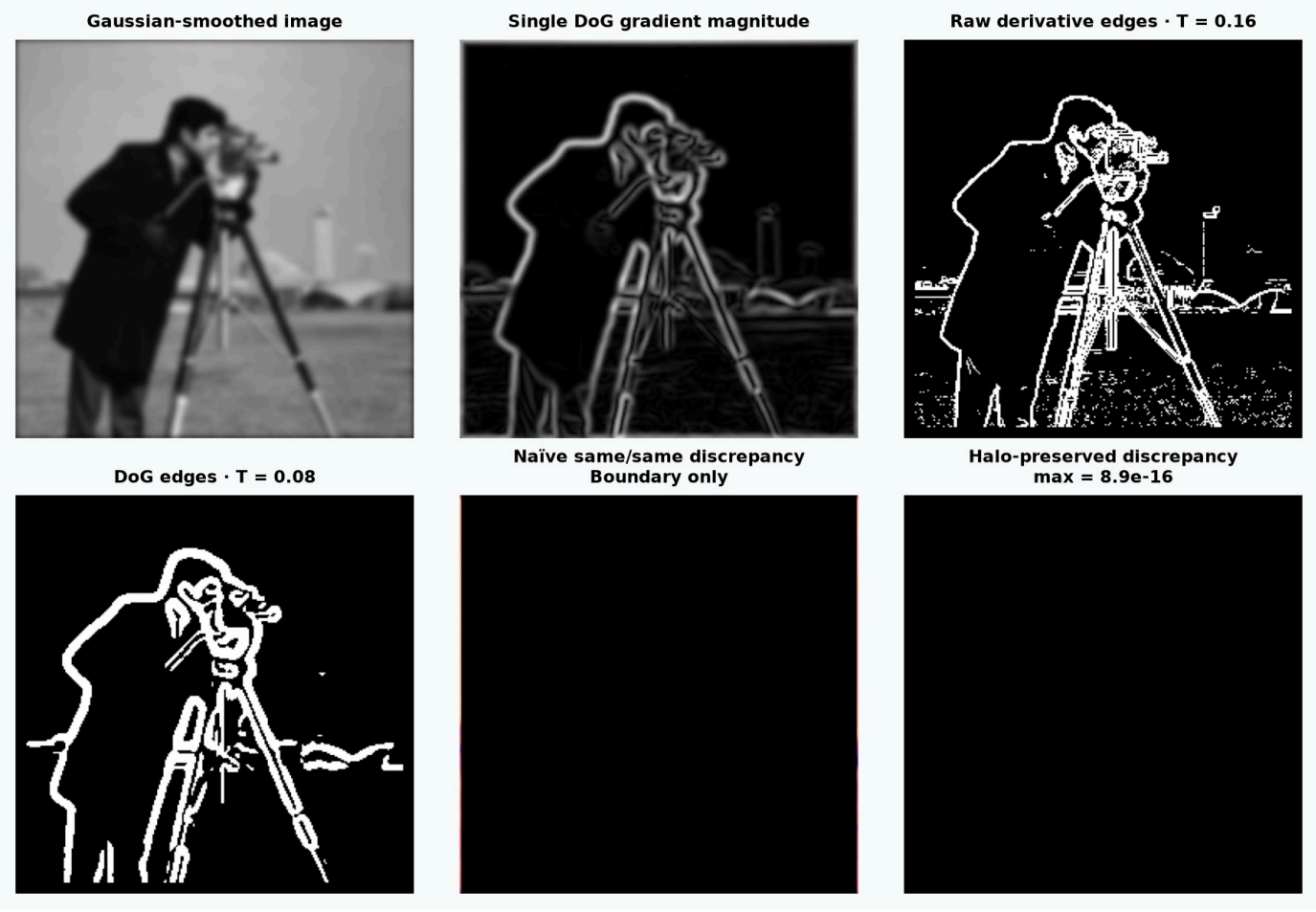
Finite differences after Gaussian smoothing, using two successive “same” convolutions with zero padding. Signed derivatives share [−0.35, 0.35]; magnitude uses [0, 0.35].

Compared with the unsmoothed derivatives, the contours are wider and the grass texture is weaker. Nearby edges can merge, and weak detail can disappear.

1.3 Sequential and combined filtering

Sequential and combined filtering should agree when both use the same boundary treatment.

Associativity $I * (G * D) = (I * G) * D$



Gaussian derivative results, thresholded edges, and boundary differences. Keeping the full intermediate blur restores agreement.

Agreement between the two formulations

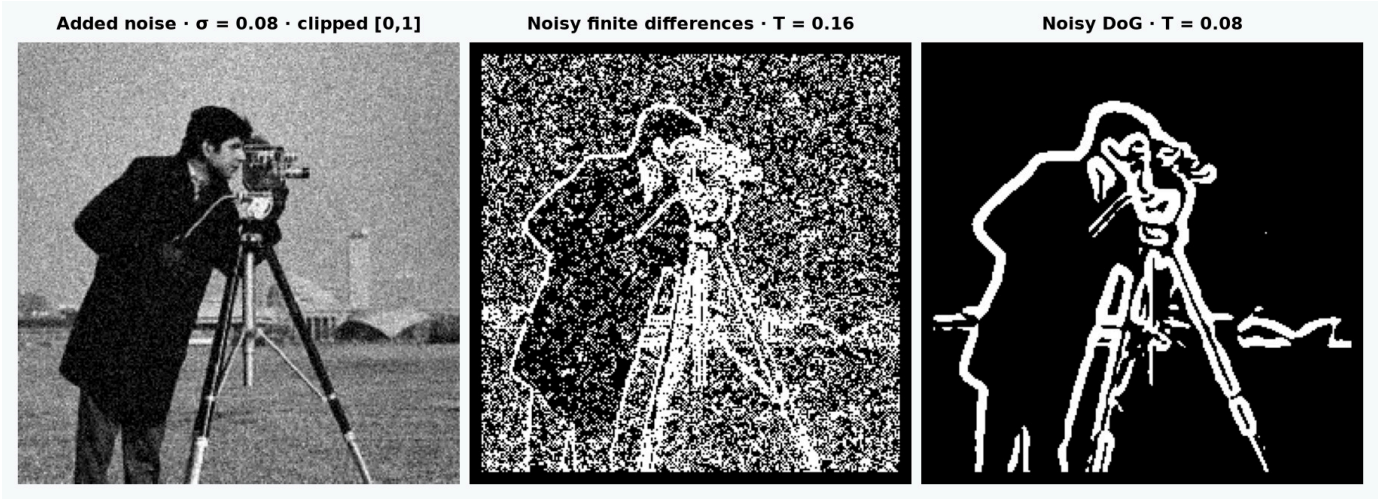
Measurement	Value
Two successive “same” convolutions vs. DoG · maximum error	0.307441
Two successive “same” convolutions vs. DoG · interior maximum error	8.95e-16
Full intermediate image vs. DoG · maximum error	8.95e-16

Two successive “same” convolutions crop the intermediate blur before differentiation. This removes samples outside the original frame and creates the boundary differences above. Keeping the full intermediate image and cropping once restores agreement across the complete output.

The edge maps use T = 0.08 for the Gaussian derivative and T = 0.16 for the original finite differences. Smoothing reduces response amplitudes, so these are separately selected visual settings, not a comparison of measured detection accuracy.

1.3 Sensitivity to added noise

Gaussian noise with σ = 0.08 is added using a fixed random seed, and the resulting intensities are clipped to [0, 1]. Each noisy gradient is compared with the same method applied to the clean image.



Cameraman image with added Gaussian noise and the resulting edge maps. Gaussian smoothing suppresses many of the noise responses visible with finite differences.

Noise residual $\text{RMS} = \sqrt{\text{mean}[(g_{x,\text{noisy}} - g_{x,\text{clean}})^2 + (g_{y,\text{noisy}} - g_{y,\text{clean}})^2]}$

Interior vector-gradient residual

Measurement	Value
Finite-difference gradient residual RMS	0.152265
DoG gradient residual RMS	0.010304

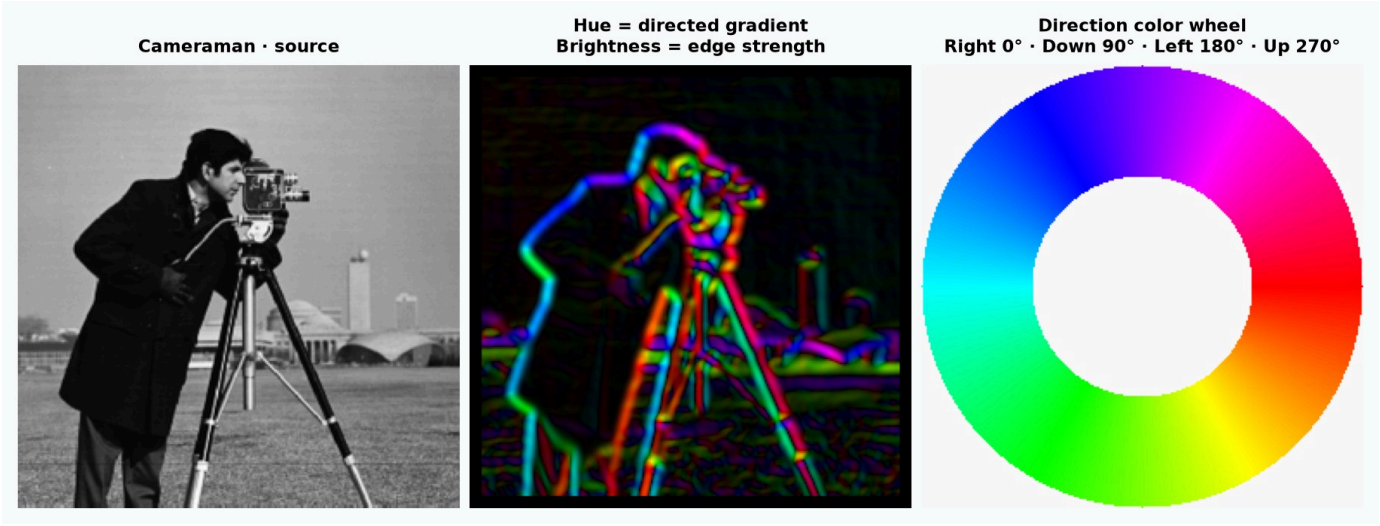
Effect of smoothing

The residual RMS decreases from 0.15226 for finite differences to 0.01030 for the derivative of Gaussian, a factor of 14.8. Smoothing suppresses many isolated noise responses, leaving a more continuous outline of the person and camera.

This test measures sensitivity to the added noise in this image. It does not measure recovery of detail lost through smoothing.

1.4 Gradient orientations

The Gaussian-derivative responses also define the direction of increasing intensity. Hue shows this direction, while brightness shows gradient strength.



Gradient orientation and direction color wheel. Hue represents direction in image coordinates; brightness represents edge strength. The seven-pixel boundary margin is excluded.

Reading the orientation map

The gradient is perpendicular to an edge tangent. Image rows increase downward, so 90° points down. Weak responses are dark, and the excluded seven-pixel boundary is black.

HSV mapping

$$h = (\theta \bmod 2\pi)/(2\pi) \quad s = 1$$
$$v = \min(|\nabla I| / 0.2386, 1)^{0.7}$$

The brightness reference, 0.2386, is the 99.5th percentile of interior gradient magnitude.

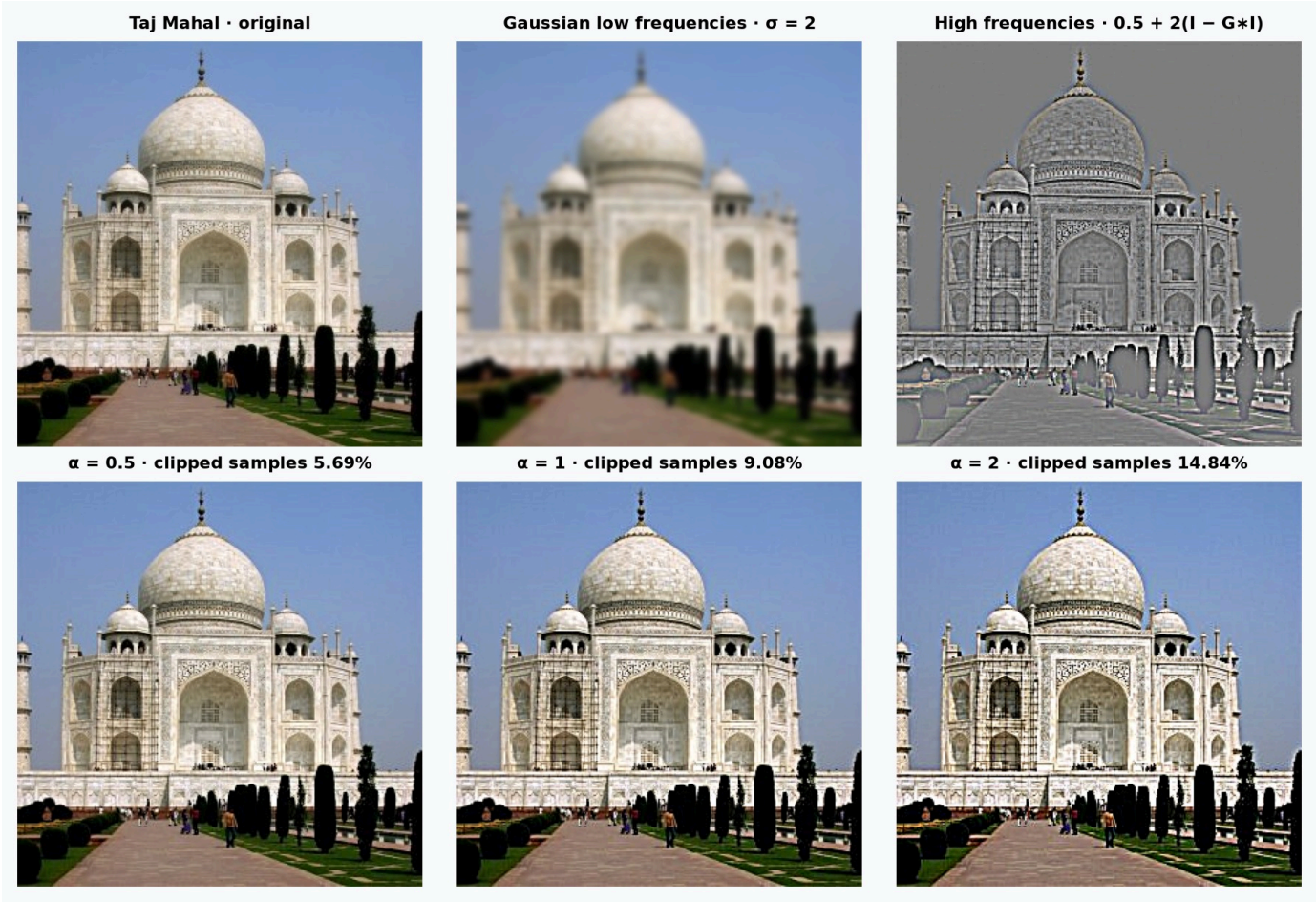
Computing the angle

The implementation uses an 18-term arithmetic series with range reduction and quadrant correction. It does not call arctangent, atan2, or an angle routine. Comparison with NumPy atan2, used only for validation, gives a maximum error of 4.44×10^{-16} radians. Appendix C gives the derivation and complete implementation.

2.1 Image sharpening

Unsharp masking adds a scaled high-frequency residual to the original image. The Gaussian blur supplies the low-frequency component; subtracting it from the input isolates detail.

Unsharp mask $B = G * I \quad H = I - B \quad S = I + \alpha H$



Taj Mahal: original image, Gaussian blur, high-frequency residual, and sharpening at α = 0.5, 1, and 2. The residual is displayed as 0.5 + 2H. Final display values are clipped to [0, 1].

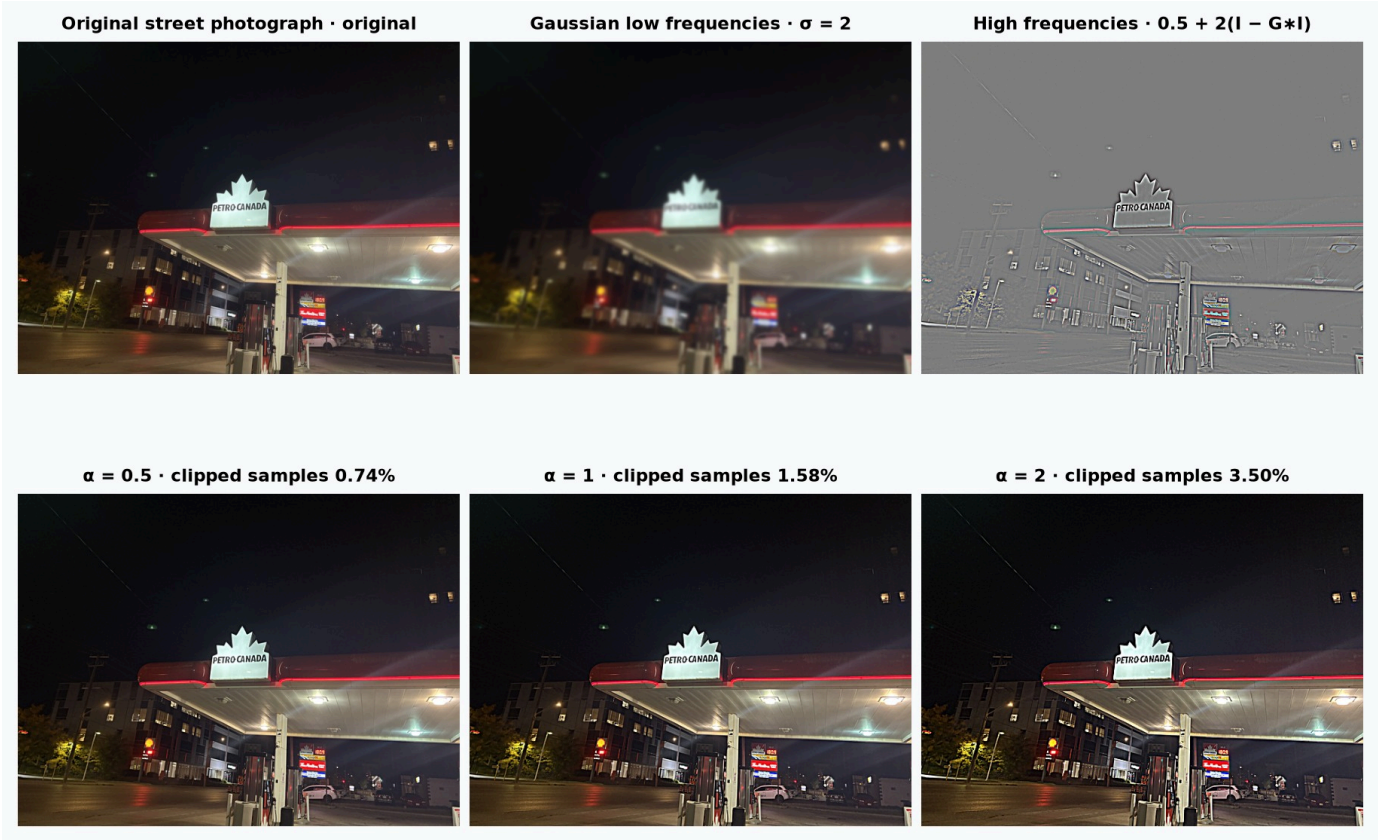
Effect of the sharpening amount

Increasing α strengthens the Taj Mahal's edges and texture. At the largest setting, halos and clipped highlights also become more apparent. Sharpening increases local contrast but can amplify noise and existing artifacts.

Both sharpening examples use a 13 × 13 Gaussian with σ = 2 pixels and symmetric boundaries. RGB channels are processed independently in float64. Residuals use the fixed display mapping 0.5 + 2H, so middle gray means zero.

2.1 Sharpening the street photograph

The same filter and sharpening amounts are applied to the original street photograph. Fine building edges and road texture become more distinct as α increases.



Original street photograph: original image, Gaussian blur, high-frequency residual, and sharpening at α = 0.5, 1, and 2. The residual is displayed as 0.5 + 2H. Final display values are clipped to [0, 1].

RGB values outside the display range

Image	α	Values outside [0, 1]
Taj Mahal	0.5	5.687%
Taj Mahal	1.0	9.075%
Taj Mahal	2.0	14.845%
Original street photograph	0.5	0.741%
Original street photograph	1.0	1.581%
Original street photograph	2.0	3.504%

The table counts scalar RGB values outside [0, 1] before export. The Taj Mahal image clips more heavily than the street image at every tested setting. Signed residuals and out-of-range values are retained during computation; only display exports are clipped.

2.1 One convolution and its limits

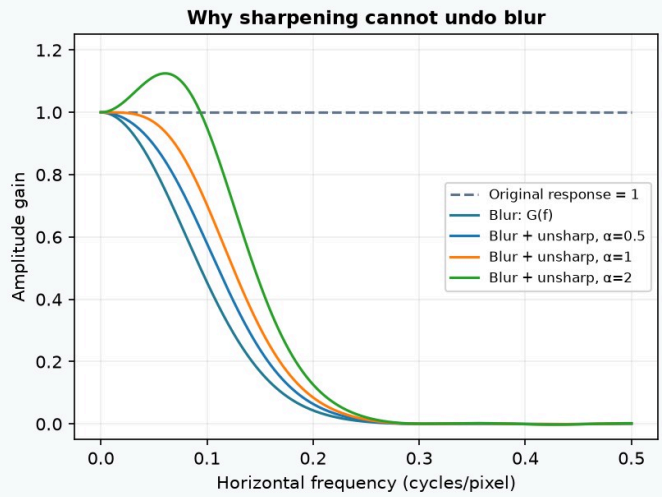
Expanding the unsharp-mask equation gives an equivalent single convolution. The unit impulse δ retains the original image, while the Gaussian term subtracts the blurred component.

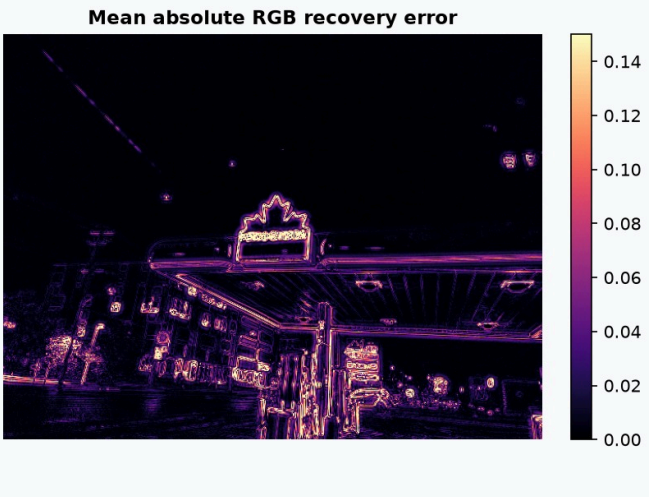
Sharpening kernel

$$S = [(1 + \alpha)\delta - \alpha G] * I$$
$$K = (1 + \alpha)\delta - \alpha G$$

The kernel sums to one and preserves a constant image when the boundary extension also preserves constants. Symmetric extension avoids the dark borders introduced by zero padding. At $\alpha = 1$, this formulation agrees with residual addition to a maximum error of 1.78×10^{-15} across both examples.

Why sharpening does not undo blur





Left: horizontal frequency response of the discrete Gaussian and the combined blur and sharpening operations. Right: mean absolute RGB error against the sharp reference at the selected setting.

Blur then sharpen

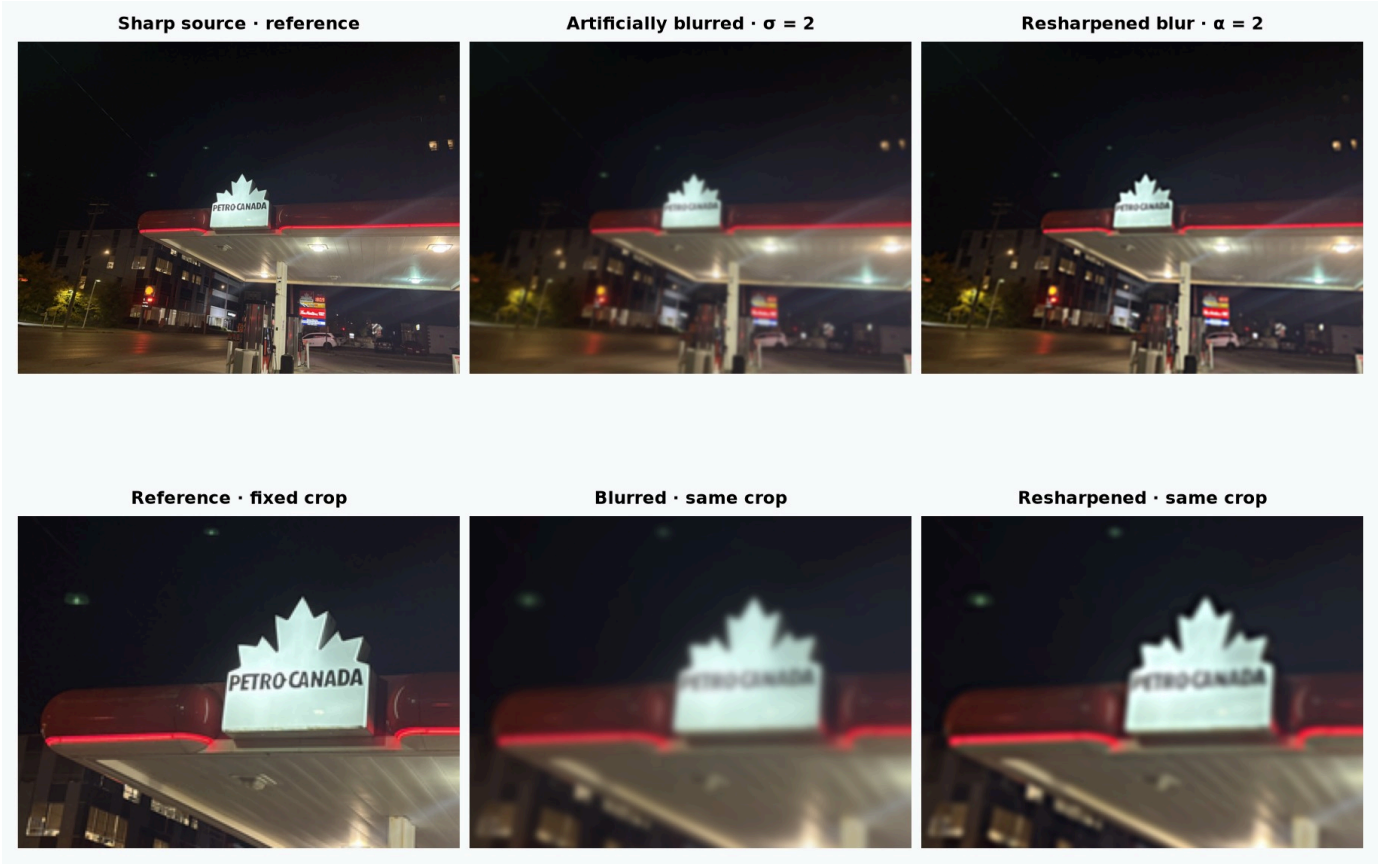
$$R(f) = G(f)[1 + \alpha - \alpha G(f)]$$

The combined frequency response generally differs from one. Frequencies strongly attenuated by the blur remain attenuated after finite amplification, and clipping can remove additional information. The nonzero error map shows that stronger edges do not imply complete recovery of the original detail.

The next comparison measures this loss against the original street photograph. Appendix D gives the single-convolution kernel code.

2.1 Blur and resharpen

The street photograph is first blurred, then sharpened with the same Gaussian. The original photograph provides a reference for the comparison.



Blur and sharpening of the original street photograph. The lower row shows the same fixed region from each image. α = 2 gives the lowest clipped RGB MSE among the three tested settings.

Recovery relative to the original street image · RGB reference in [0, 1]

Stage	α	Unclipped MSE	Unclipped PSNR (dB)	Clipped MSE	Clipped PSNR (dB)
Original	—	0	∞	0	∞
Blurred	—	0.001660	27.80	0.001660	27.80
Resharpened	0.5	0.001429	28.45	0.001429	28.45
Resharpened	1.0	0.001259	29.00	0.001259	29.00
Resharpened	2.0	0.001101	29.58	0.001099	29.59

Of the three tested amounts, α = 2 gives the lowest clipped RGB MSE: 0.001099, compared with 0.001660 after blurring. PSNR rises from 27.80 to 29.59 dB. This setting is selected against the original reference and applies to this experiment.

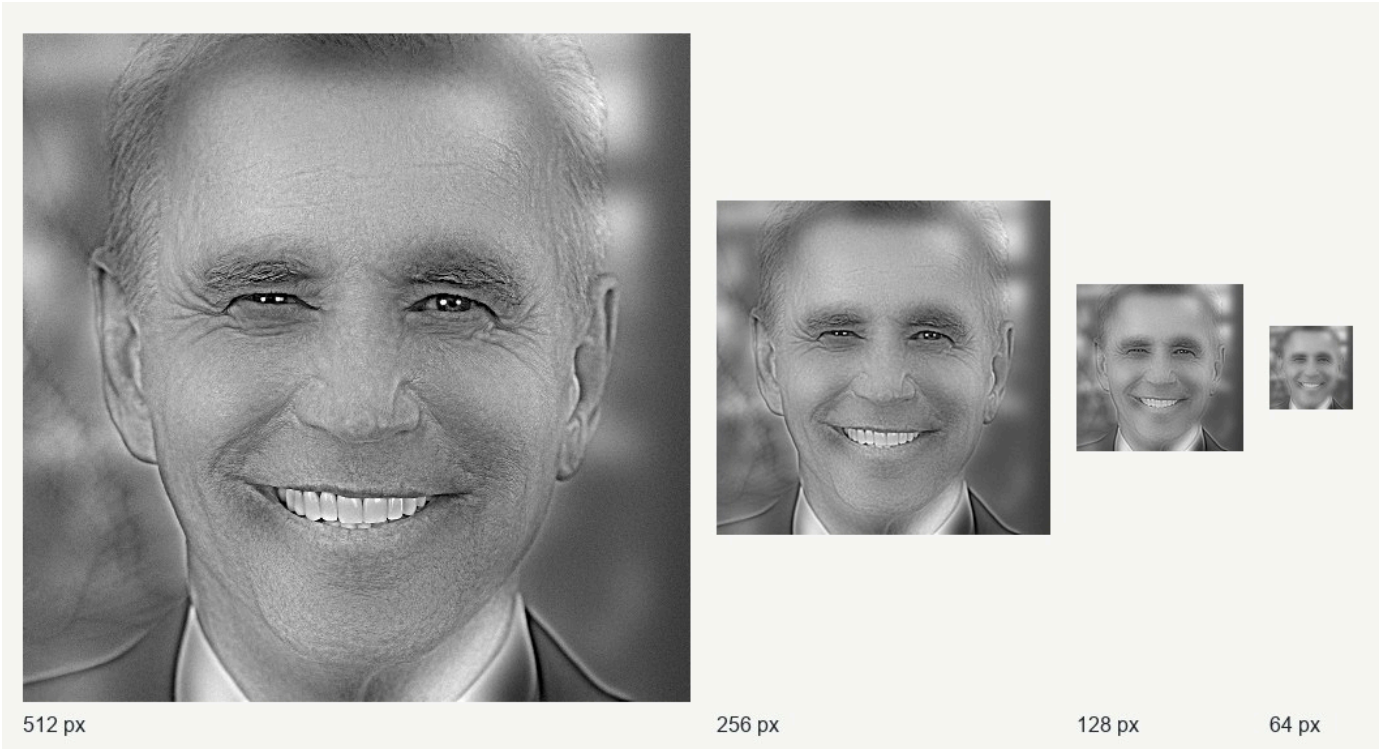
MSE is averaged over all RGB values. With a reference range of [0, 1], PSNR = 10 $\log_{10}(1/\text{MSE})$, in decibels. Both unclipped and display-clipped results are reported.

2.2 Hybrid images at different scales

A hybrid image combines broad structure from one image with fine detail from another. The supplied Derek–Nutmeg example is followed by two custom portrait pairs: Obama–Biden and the classic Monroe–Einstein combination.

Hybrid construction $I_{hybrid} = G_{\sigma L} * A + g(B - G_{\sigma H} * B)$

A is the low-frequency source, B is the high-frequency source, g is the detail gain, and Gσ is a Gaussian kernel with standard deviation σ. The symbol ∗ denotes convolution.



Obama + Biden: the same hybrid at 512, 256, 128 and 64 pixels, reduced with Lanczos.

Biden's eyes, wrinkles and fine facial detail are prominent in the large view. At 64 pixels, Obama's broader facial structure becomes clearer. The intermediate views retain both identities; the transition is gradual.

Selected settings at 512 x 512 pixels

Pair	Low-pass sigma (px)	High-pass sigma (px)	Gain	Values clipped
Derek + Nutmeg	10	5	2	2.83%
Obama + Biden	12	3	2	0.17%
Monroe + Einstein	12	3	1.25	0.35%

For a distance comparison, keep the largest image at a fixed display size and move farther away. Reduced versions provide another comparison; screen size, zoom and eyesight prevent assigning a calibrated viewing distance.

2.2 Choosing filter widths and gain

The low-pass width controls the broad portrait structure. Increasing the high-pass width retains more middle-scale detail, while gain changes its contrast. The selected settings balance a distinct near view against a clear reduced view.



Obama + Biden: filter-width and gain trials, synthesized at 512 x 512 and displayed at 256 and 64 pixels. The selected panel uses the final settings. Alignment remains fixed across the trials.

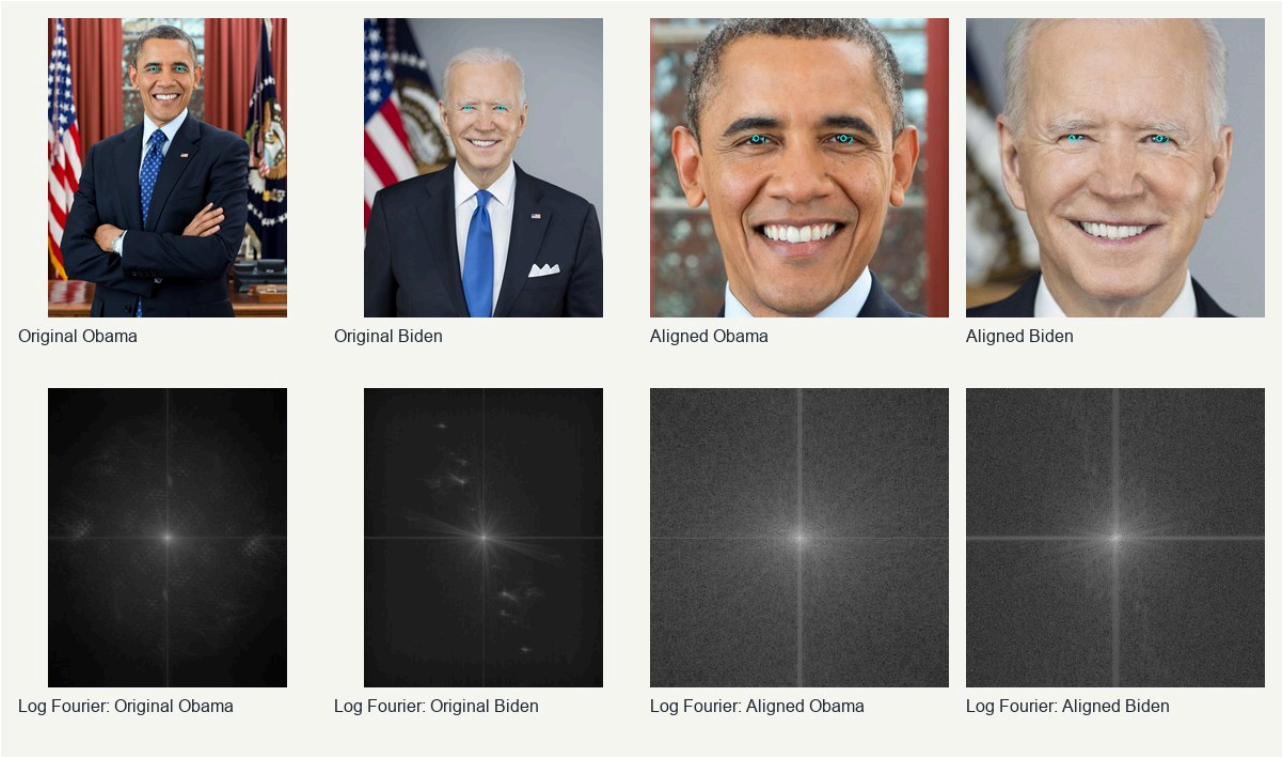
What changes in the trials

The six trials keep eye alignment fixed. Lower gain weakens Biden's wrinkles and hair detail. Increasing the low-pass width to 20 blurs Obama's eyes and smile. High-pass σ = 7 retains more of Biden's face in the reduced view, making the identity change less distinct.

Selected (σL, σH, g) settings are Derek: (10, 5, 2); Obama + Biden: (12, 3, 2); Monroe + Einstein: (12, 3, 1.25). The choices are visual judgments for these sources. The later controlled Derek–Nutmeg comparison separates the effects of geometry and filtering.

2.2 Selecting and aligning the portraits

Obama and Biden have similar near-frontal poses, open smiles and collared clothing. Monroe's softly lit portrait reduces mouth and neck shadows. Mirroring Einstein horizontally brings the facing directions and cheek shading into closer agreement.



Original and aligned Obama and Biden inputs. Cyan circles show the alignment landmarks; spectra are calculated from the unmarked images. Both Fourier figures use the same logarithmic scale and display maximum.

Common eye alignment

Two-point similarity transforms map each pair's eyes to the common targets below. No separate face scaling or local deformation is applied.

Low-source transform relative to each pair's common eye alignment.

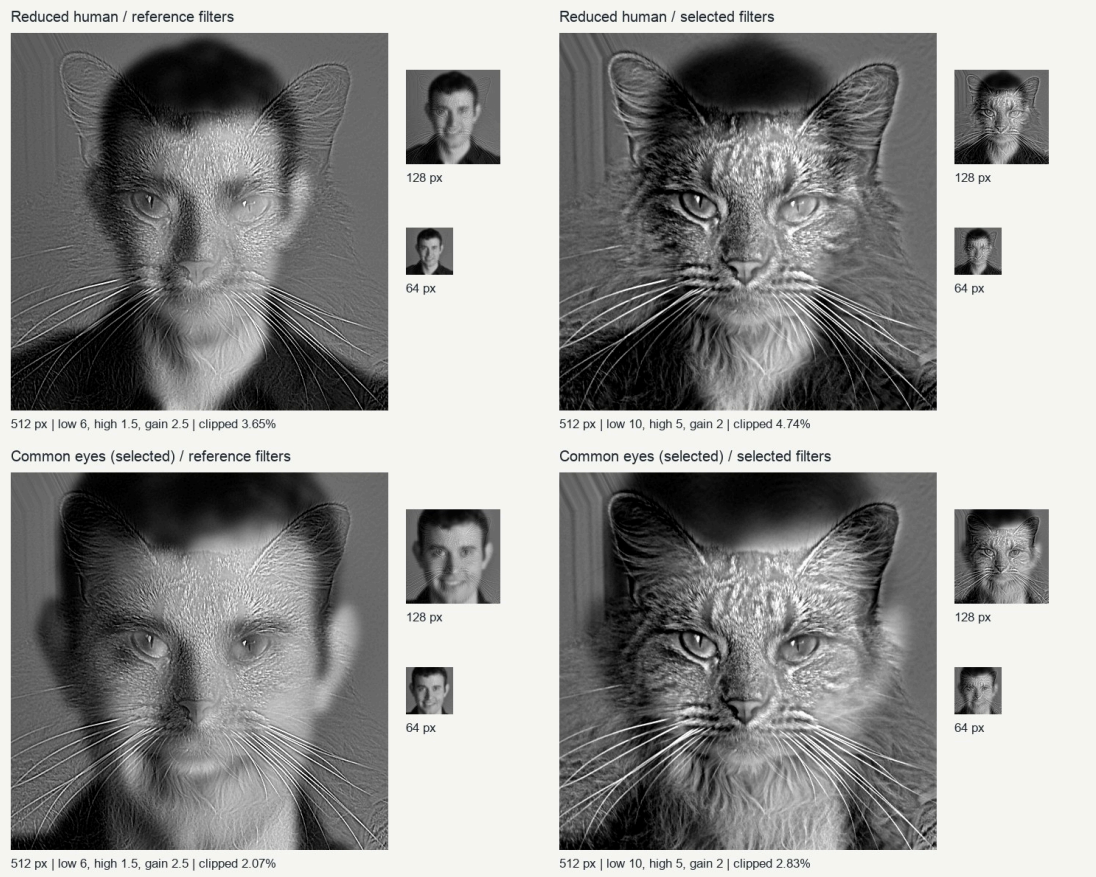
Pair	Scale	Vertical shift (px)	Low-source eye centers (px)
Derek + Nutmeg	1.00	0	(186, 235); (326, 235)
Obama + Biden	1.00	0	(181, 205); (331, 205)
Monroe + Einstein	1.00	0	(191, 215); (321, 215)

Derek and Nutmeg have different proportions. The next page compares common eye alignment with a smaller Derek face, keeping Nutmeg's framing fixed.

Einstein's reflection reverses x as W–1–x and swaps the eye correspondence. It is composed with the similarity transform; each original is sampled once, using bilinear interpolation and replicated borders. Uniform scaling preserves proportions. Original landmarks and exact matrices are saved with the results.

2.2 Separating geometry and filtering

For Derek + Nutmeg, the four trials vary human position and filter settings independently. The reduced-human alternative uses the selected cat framing; it does not reproduce an earlier crop. The cat transform, photographs, color treatment and interpolation remain fixed.



Rows compare reduced-human and selected common-eye geometry; columns compare reference and selected filters. Each trial is shown at 512, 128 and 64 pixels.

Geometry and filtering varied independently for Derek + Nutmeg.

Geometry	(σL, σH, g)	R_{128}	C (%)
Reduced human	(6, 1.5, 2.5)	0.163	3.65
Reduced human	(10, 5, 2)	0.478	4.74
Common eyes (selected)	(6, 1.5, 2.5)	0.150	2.07
Common eyes (selected)	(10, 5, 2)	0.438	2.83

R_{128} is the high-to-low luminance standard-deviation ratio after signed Lanczos reduction to 128 pixels; the high band includes gain g. C is the percentage of scalar channel values outside [0, 1] before display clipping.

On common eyes geometry, changing from reference to selected filters changes clipping from 2.07% to 2.83%. With selected filters fixed, changing from reduced human to common eyes changes clipping from 4.74% to 2.83%. These values do not measure visual quality. Common eye alignment gives a more coherent cat eye and cheek region, while broad human hair and neck shading remains.

2.2 Frequency content of the hybrid

A Gaussian has no sharp cutoff. Its approximate half-amplitude frequency provides a reference for the selected filter widths.

Half-amplitude frequency $f_{1/2} = \sqrt{(2 \ln 2)} / (2\pi\sigma)$

For Obama + Biden, the low-pass and complementary high-pass values are 0.0156 and 0.0625 cycles per output pixel, respectively. Filtering uses spatial convolution; Fourier plots show the resulting frequency content.



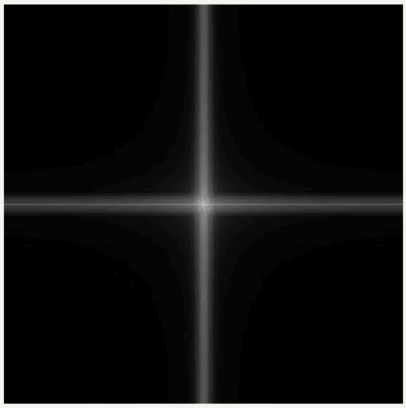
Gaussian low pass



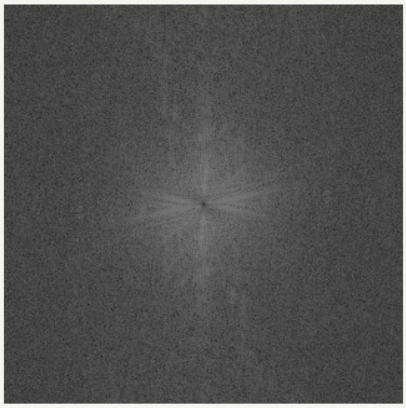
Gain-scaled signed high pass



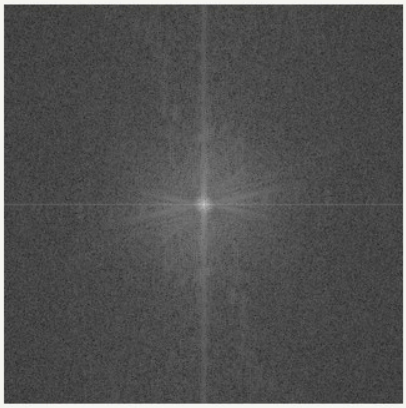
Final hybrid



Log Fourier: Gaussian low pass



Log Fourier: Gain-scaled signed high pass



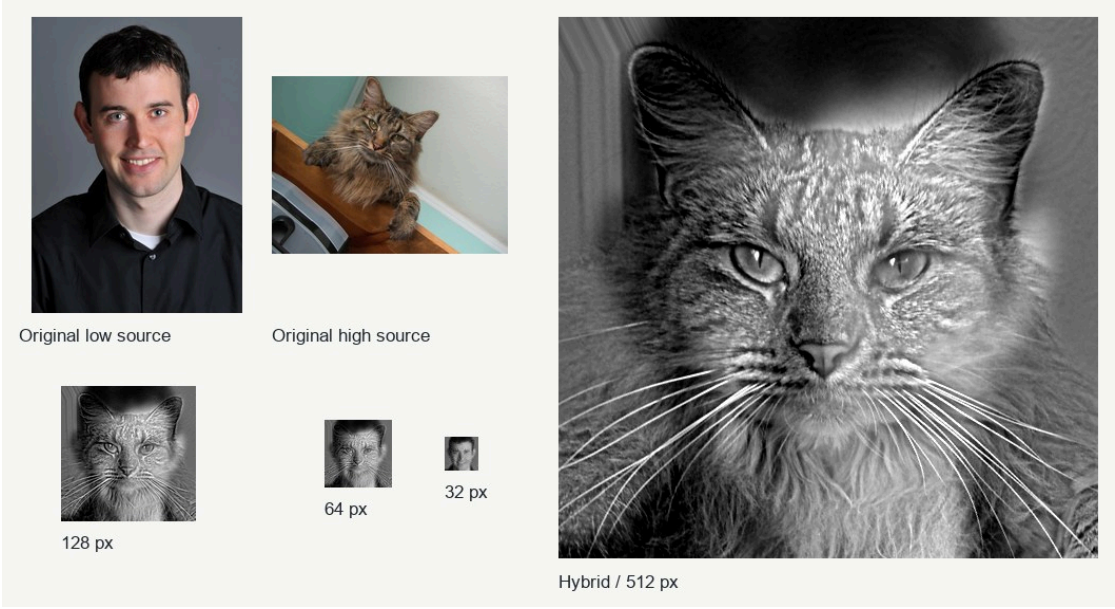
Log Fourier: Final hybrid

Low-pass energy is concentrated near the center; the high-pass suppresses central energy and retains fine detail. Adding the two components produces the hybrid. Both Fourier figures use the same logarithmic scale and display maximum.

Fourier display $\log(1 + 1{,}000{,}000\ |\text{fftshift}(\text{fft2}(Y))| / N)$

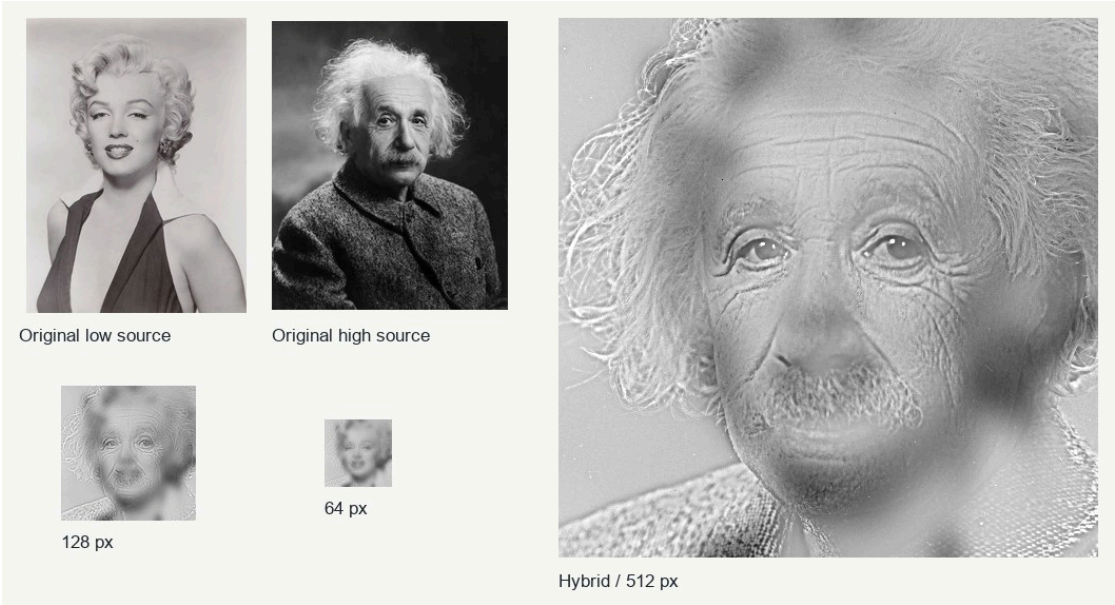
Y is luminance and N is the number of image pixels. Division by N permits comparison of differently sized inputs. The source and process spectra share one display maximum. No window or mean subtraction is applied.

2.2 Two further hybrid examples



Derek + Nutmeg: original sources and final hybrid at 512, 128, 64 and 32 pixels.

At fixed eye alignment, increasing the high-pass width from 3 to 5 and gain from 1.25 to 2 strengthens Nutmeg's eyes, muzzle and fur. Low-pass width 10 softens Derek's detail. Nutmeg dominates the large view; Derek emerges at 32 pixels. Intermediate views and broad shading remain mixed.



Monroe + Einstein: original portraits and hybrid at 512, 128 and 64 pixels; Einstein is mirrored before alignment.

Mirroring Einstein improves the facing direction and cheek-shadow match. His wrinkles and moustache dominate the large view; Monroe's features emerge at 64 pixels. Jaw proportions and some facial shading remain mixed. This implements the classic pairing [5].

The online supplement includes alignment, filter trials, geometry controls and the reflection comparison. The Earth–Moon hybrid remains a limitation study: its similar outlines do not yield a clear subject change.

Additional parameter trials and diagnostic figures are available in the online supplement.

2.2 Comparing the frequency bands

Two diagnostics describe the selected hybrids and a common baseline. The band-contrast ratio measures the relative variation in the two components; the clipping fraction records how much of their sum exceeds the display range.

Relative band contrast at width s $R_s = \mathrm{std}(D_s(H)) / \mathrm{std}(D_s(L))$

L is low-pass luminance and H is gain-scaled high-pass luminance. D_s reduces each signed component to width s with Lanczos interpolation; the native 512-pixel arrays are unchanged. Standard deviations use all pixels. Neither component is clipped or contrast-normalized.

Strict clipping fraction $C = \mathrm{count}(I_{hybrid} < 0 \text{ or } I_{hybrid} > 1) / \mathrm{count}(\text{all channel values})$

The same diagnostics for all three hybrid pairs.

Pair	Setting	R_{512}	R_{128}	C (%)
Derek + Nutmeg	Baseline	0.297	0.219	1.00
Derek + Nutmeg	Selected	0.594	0.438	2.83
Obama + Biden	Baseline	0.283	0.235	0.01
Obama + Biden	Selected	0.440	0.294	0.17
Monroe + Einstein	Baseline	0.501	0.387	0.32
Monroe + Einstein	Selected	0.526	0.333	0.35

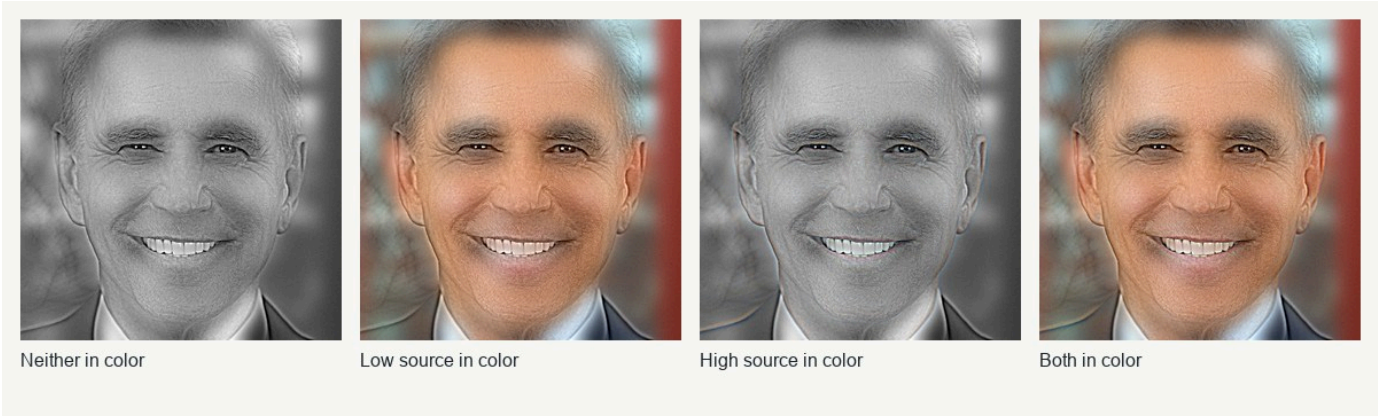
The baseline uses (σL, σH, g) = (10, 5, 1) on each pair's selected geometry. Selected settings are listed with the hybrid results. This baseline differs from the (6, 1.5, 2.5) reference used in the preceding geometry comparison.

For all three selected pairs, R decreases from 512 to 128 pixels: the fine-detail band loses contrast relative to the broad low-frequency band. The baseline provides the same measurements on the same geometry, allowing the filter choices to be compared without changing alignment.

These measurements describe the bands, not identity recognition. All selected pairs use common eye targets. Derek's geometry comparison also tests a reduced-human alternative, while the two custom pairs use similarly posed portraits. All retain mixed intermediate views. A larger ratio or smaller clipping fraction alone cannot establish a cleaner change in perceived subject.

2.2 Color in the frequency bands

Four variants retain color in neither source, only the low-frequency source, only the high-frequency source, or both. Alignment, filter widths and gain remain identical.



Four color variants of Obama + Biden with identical alignment, filter widths and gain. Luminance conversion uses weights 0.2126, 0.7152 and 0.0722. Low-frequency color preserves Obama's broad skin, hair and clothing regions; high-frequency color remains mainly around Biden's fine facial and hair edges.

How color changes the result

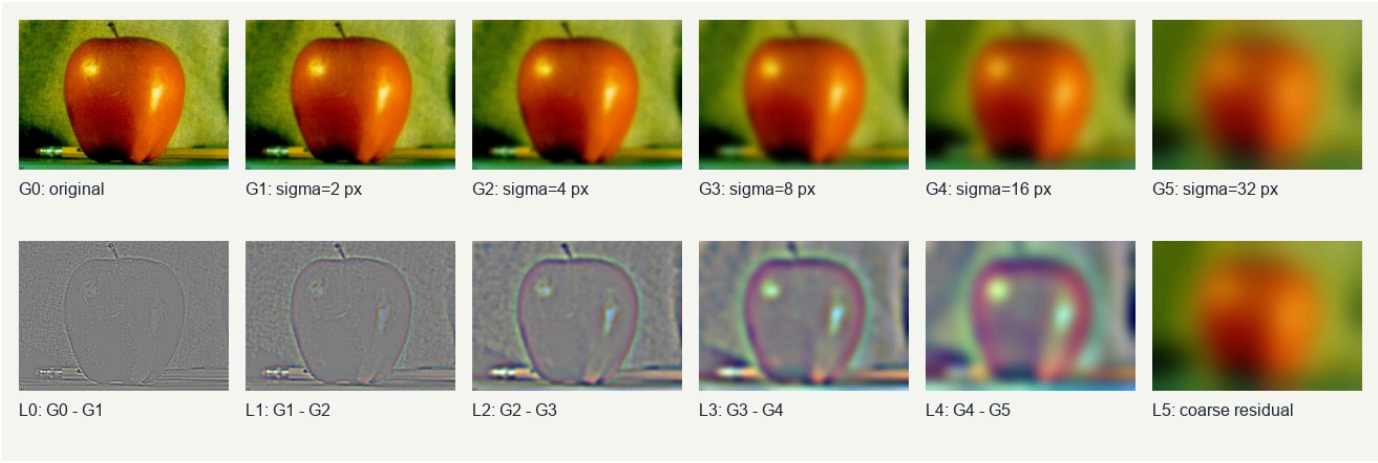
Low-frequency color preserves Obama's broad skin, hair and background colors, agreeing with the portrait visible at smaller sizes. Biden's fine detail remains visible in the larger view. High-frequency color contributes mainly near edges because the broad color regions have been removed.

Keeping both sources in color adds some competing edge colors. The main examples use grayscale so that the change in facial structure can be assessed without this broad color difference.

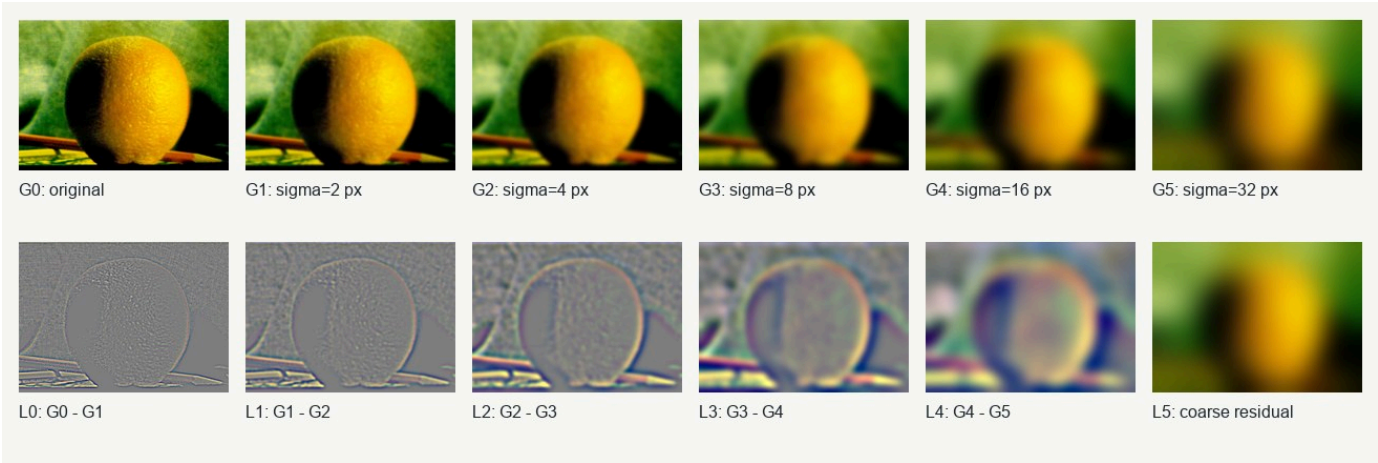
2.3 Gaussian and Laplacian stacks

The supplied burt_apple.png and burt_orange.png images are each 740 × 530 pixels. One resize with area interpolation gives 512 × 367 working images, reducing computation while preserving aspect ratio to the nearest output pixel.

Every Gaussian-stack level keeps the same dimensions. G_0 is the original; G_1 through G_5 independently blur it at σ = 2, 4, 8, 16 and 32 pixels. Each width therefore refers to the original image. Construction is explicit, with no downsampling or built-in pyramid routine.



Apple: Gaussian stack above and Laplacian stack below, including the coarse residual. Every level retains the working resolution of 512 × 367 pixels. Signed detail bands use 0.5 + 2.5L for display; the residual uses its actual intensity.



Orange: Gaussian stack above and Laplacian stack below, including the coarse residual. Every level retains the working resolution of 512 × 367 pixels. Signed detail bands use 0.5 + 2.5L for display; the residual uses its actual intensity.

Laplacian detail and residual $L_i = G_i - G_{i+1}$ $(i = 0, \ldots, 4)$; $L_5 = G_5$

The five detail bands sum to the original minus G_5. Adding the coarse residual restores large-scale brightness and color, reconstructing the image up to floating-point rounding.

2.3 Reconstructing the oraple

The selected levels show how each source contributes at different scales. The final row sums all five detail bands and the coarse residual.

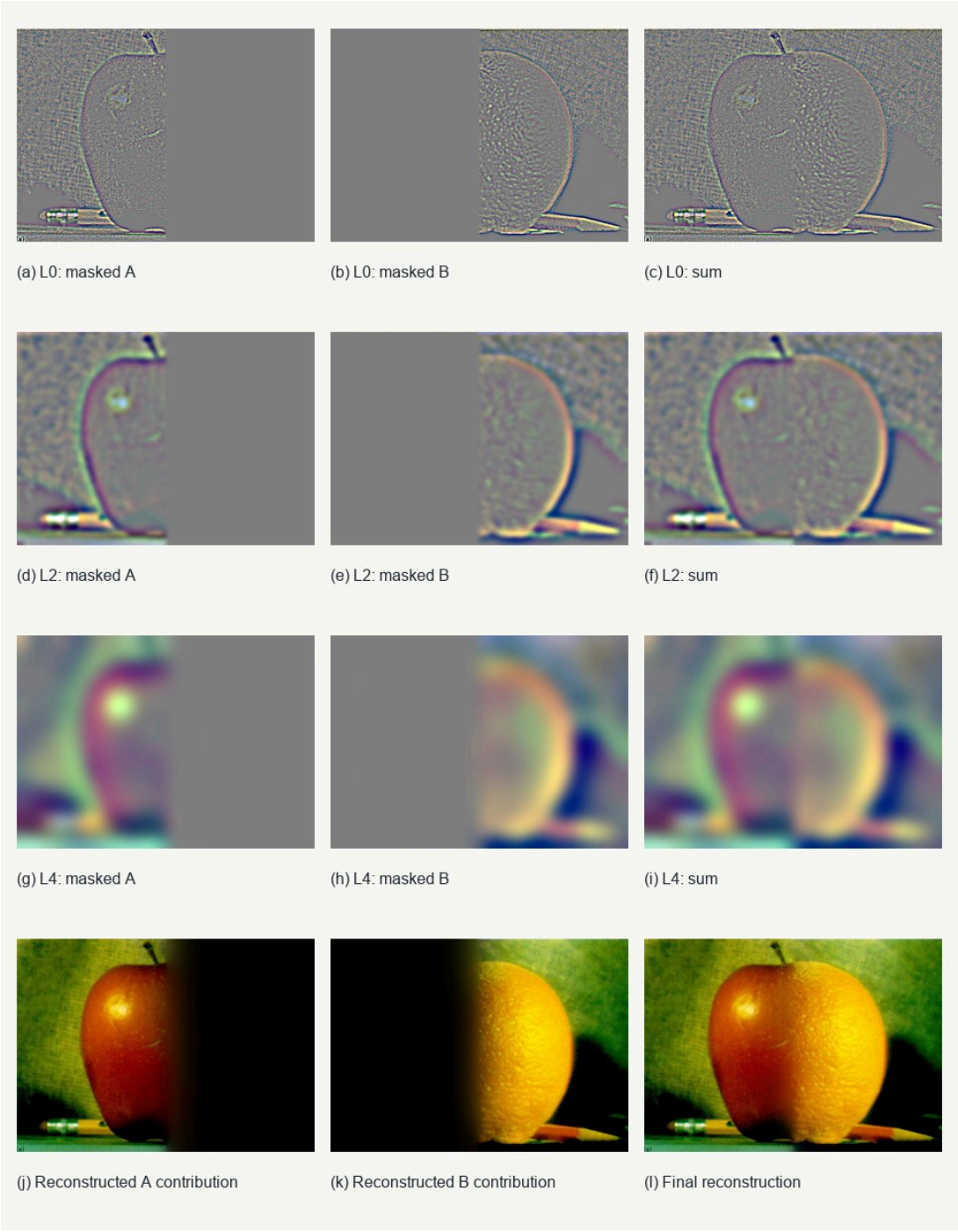


Figure 3.42 (a)–(l) layout: rows 1–3 show levels 0, 2, and 4; columns show the masked apple, masked orange, and their sum. Row 4 sums all five detail bands and the coarse residual to reconstruct the two contributions and final oraple. Detail bands share a symmetric display scale within each row; reconstructed images use actual [0, 1] intensity.

2.4 A transition that changes with scale

A binary mask selects apple on the left and orange on the right. Its Gaussian stack uses the same smoothing scales as the image stacks.

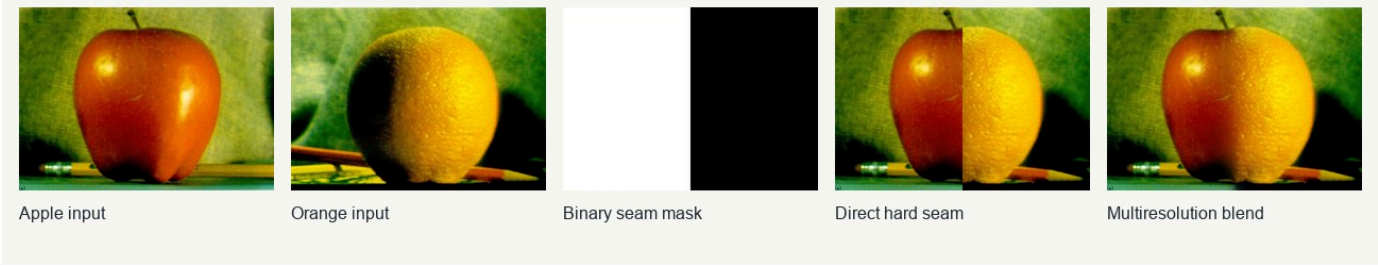


Gaussian stack of the left/right selection mask, using the same scales as the image stacks. The transition widens at coarser levels. White selects apple and black selects orange.

Blend each band, then sum

$$S_i = M_i L_{A,i} + (1 - M_i) L_{B,i}; \quad I_{blend} = \Sigma_{i=0}^{5} S_i$$

The rule includes the coarse residual. The finest mask has a sharp transition; coarser masks spread brightness and color changes across wider regions while retaining local texture.



Supplied inputs, selection mask, direct binary cut, and multiresolution oraple. Both methods use the same 512 × 367 images with preserved aspect ratio. The seam lies between zero-based columns 255 and 256.

Both methods use the same inputs and initial mask. The direct cut leaves a visible vertical change in brightness and color. The mean absolute RGB difference across adjoining seam columns falls from 0.1716 to 0.0279 with multiresolution blending. This measures the local discontinuity, not overall perceptual quality.

Reconstruction accuracy

Comparison	Maximum absolute error
Apple reconstruction	1.110e-16
Orange reconstruction	2.220e-16
Final image vs. sum of mixed bands	3.331e-16

The seam lies between zero-based columns 255 and 256. Measurement uses the floating-point reconstruction before clipping. Stack construction and reconstruction remain unclipped; display export limits 2.35% of scalar RGB values to [0, 1].

2.4 Adding the bands one level at a time

Starting with the mixed coarse residual S_5, add the five detail contributions from coarse to fine. Every addition uses arrays of the same size; no interpolation or resizing occurs.

Cumulative reconstruction $T_5 = S_5; \quad T_k = T_{k+1} + S_k, \quad k = 4, \ldots, 0$



Read left to right, then top to bottom. All panels use the same [0, 1] display limits, with no contrast normalization. Measurements use the signed arrays before clipping.

All levels remain 512 × 367 RGB; σ is in output pixels.

Step	Added band	Mask σ	Remaining RMS
1	S5	32	0.10923
2	S4	16	0.07960
3	S3	8	0.05434
4	S2	4	0.03644
5	S1	2	0.02469
6	S0	0	3.25e-17

Remaining RMS = $\sqrt{\text{mean}[(I_{\text{blend}} - T_k)^2]}$, over all pixels and RGB channels. The reference is the complete computed blend, not an ideal photograph. The final difference is floating-point roundoff; intermediate RMS need not decrease for arbitrary signed bands.

Omitting S_5 leaves RMS error 0.39660: the detail bands cannot restore broad brightness and color. Including it reconstructs the full blend to numerical precision. This checks completeness of the summation, not visual quality.

2.4 An alpine lake through a sandstone arch

North Window's orange sandstone frames Lake McDonald and its snowy peaks. The source photographs contribute visibly different content. Their warm light provides a compatible starting point, while the composite deliberately joins two different places.



Original A: Alpenglow on Lake McDonald | Original B: North Window sandstone arch | Framed A: 512 x 512 | Framed B: 512 x 512

Sandstone arch and alpine lake: original and framed photographs. Cyan circles mark independent framing controls used for cropping and uniform scaling; they are not correspondences between the two places. The lake is A, and the sandstone arch is B. North Window arch: Thomas Wolf, www.foto-tw.de, CC BY-SA 3.0. Lake McDonald: David Restivo/NPS (GlacierNPS), CC BY 2.0. The adapted composite and arch-derived figures are distributed under CC BY-SA 3.0. It joins photographs of different places and is not a record of a real scene.

Framing and selection

Independent framing points crop and uniformly scale each photograph to 512 × 512. They are not corresponding features of one scene. Bilinear sampling uses replicated borders. The inputs receive no color correction or prior background transfer.

White selects lake A; black selects arch B

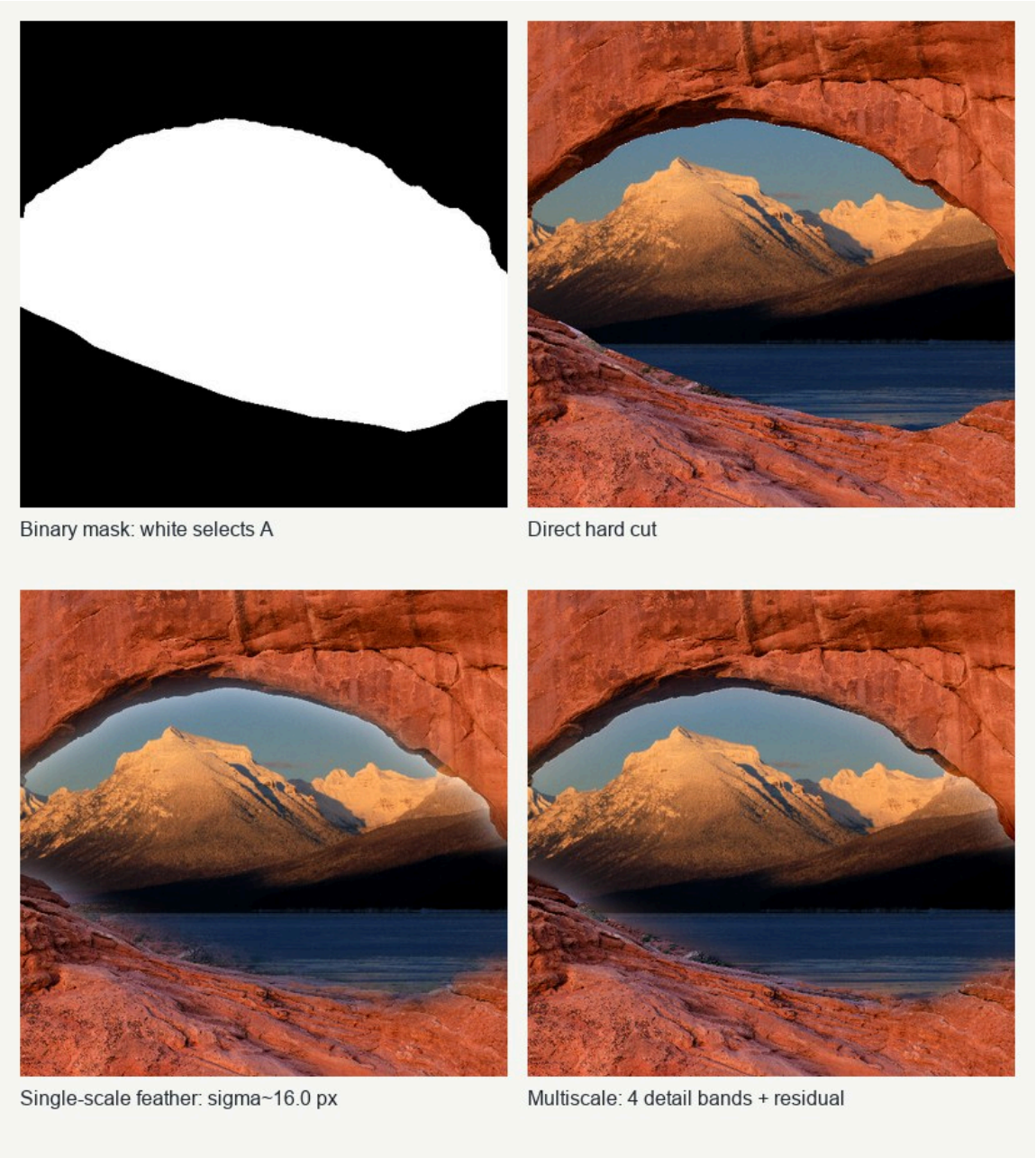
$$S_i = M_i L_{A,i} + (1 - M_i) L_{B,i}$$

A saved 122-vertex polygon follows the inner arch and the foreground ledge. Its upper boundary was traced from the framed sky edge and visually checked to remove remnants of the original sky. The lower boundary defines where the lake meets the foreground rock.

Four detail bands use image σ = 2, 4, 8 and 16 pixels, plus the coarse residual. Initial mask σ = 1 softens pixel-level contour corners. Nominal mask widths are 1, 2.2, 4.1, 8.1 and 16.0 pixels across the five levels.

2.4 Landscape blending: boundary and texture

The direct cut leaves a pasted edge along the foreground ledge. Broad feathering blends the full images over one wide transition, making rock partly transparent and introducing background detail through the foreground.

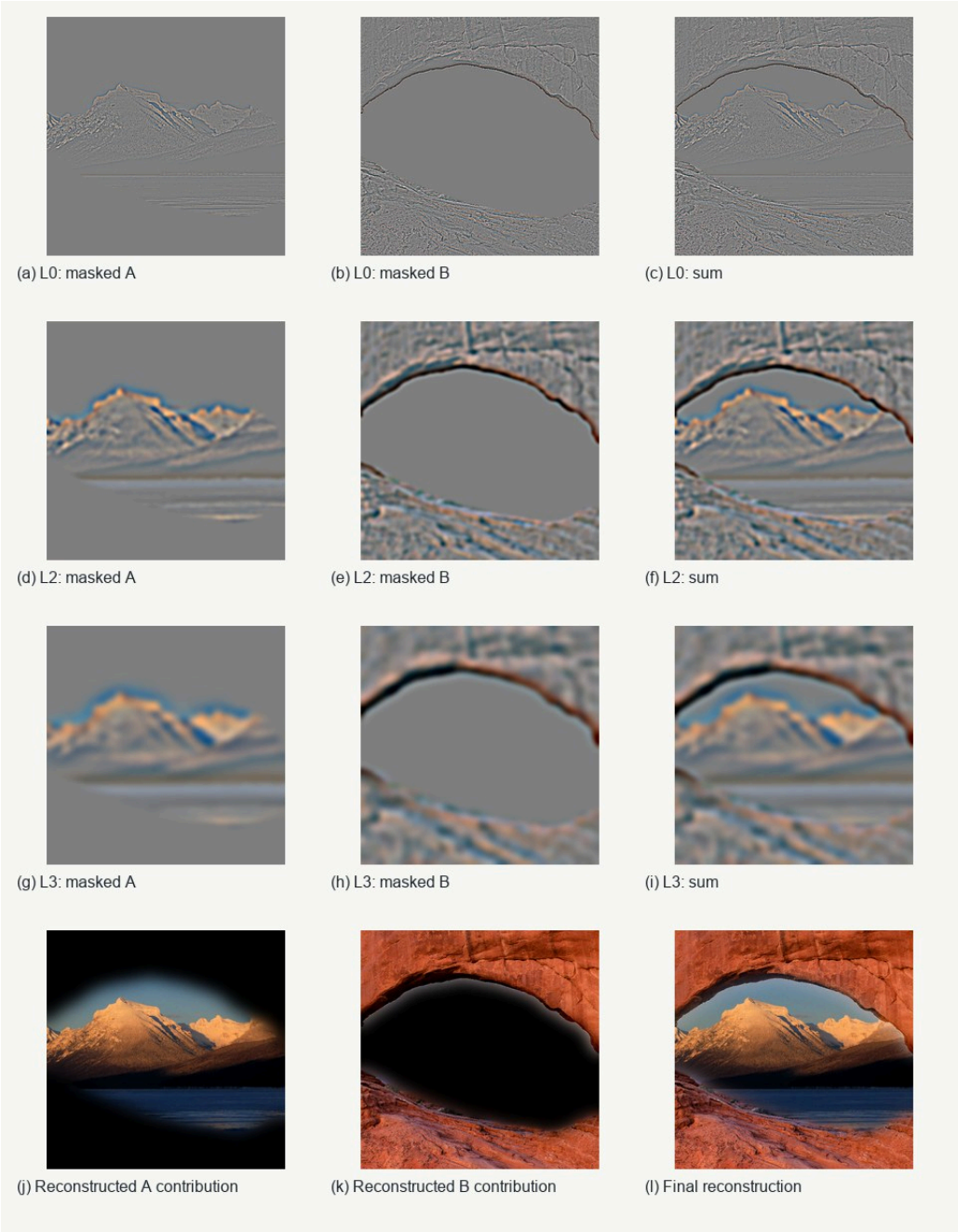


Hard cut, single-scale feathering and multiresolution blending use the same inputs and binary selection. Initial mask smoothing uses σ = 1 pixel.

Multiresolution blending retains more local rock and mountain texture while reducing the abrupt transition. Some lightening remains near the lower-left boundary. The warm peaks and arch are visually compatible, but this imagined view is not a photograph of one real location.

2.4 Landscape contributions at each scale

Three detail bands leave a tighter lower boundary. Five spread more broad color across the opening and foreground. Four bands, with maximum image σ = 16 pixels, balance seam smoothing against the loss of local contrast.

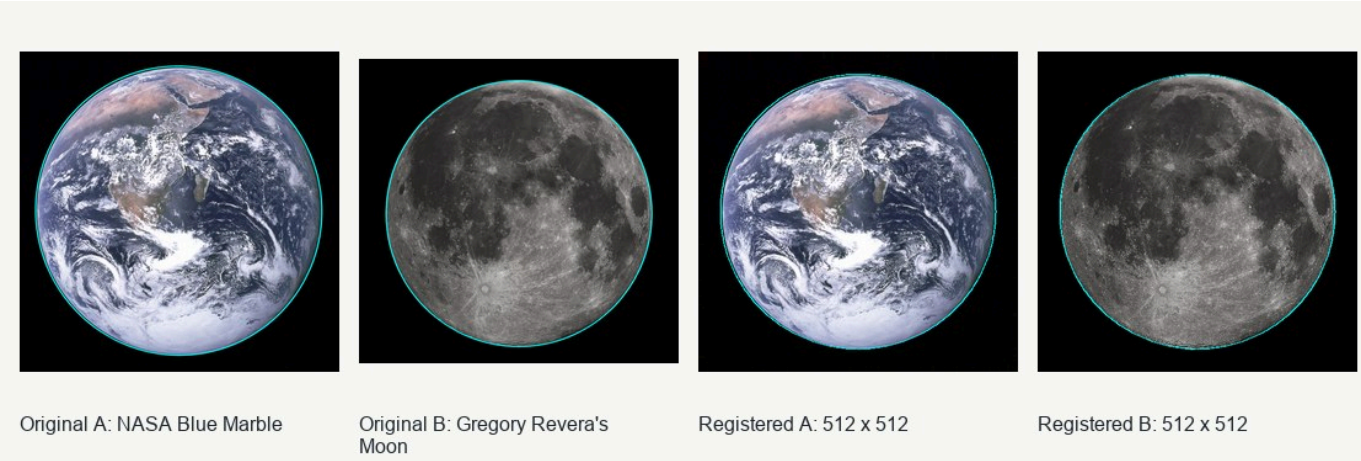


Sandstone arch and alpine lake: rows 1–3 show masked A, masked B and their sum at Laplacian detail levels 0, 2 and 3. The final row reconstructs each contribution and the result using all detail bands plus the coarse residual, following Burt–Adelson Figure 10. Each detail row uses one shared symmetric contrast scale; reconstructed rows retain actual intensity.

Rows show detail levels 0, 2 and 3, then the full reconstruction including residual level 4. Supplementary trials vary band count over 3, 4 and 5, and initial-mask σ over 0, 0.5, 1, 2, 4 and 8 pixels. Larger mask widths increasingly mix rock and lake texture; σ = 1 retains the foreground edge.

2.4 Joining Earth and Moon

An S-shaped boundary joins Earth's oceans and clouds to lunar craters. The similar disk outlines support this composite, although the same photographs did not produce a clear subject change in the hybrid experiment.



Earth-Moon blending: original and registered photographs. Cyan outlines mark the fitted source ellipses and target circle. The larger axis scale factor exceeds the smaller by 1.06% for Earth and 0.03% for the Moon. Earth: NASA/Apollo 17, public domain. Moon: Gregory H. Revera, CC BY-SA 3.0. The adapted composite and Moon-derived figures are distributed under CC BY-SA 3.0.

The largest source contours above luminance 0.04 are fitted with cv2.fitEllipse and mapped affinely to a circle of radius 220 pixels at (256, 256). Thresholding measures the outlines only; the transforms sample original RGB pixels. Earth's axis scale factors differ by 1.06%; the Moon's differ by 0.03%.

Earth–Moon uses six detail bands, adding image σ = 64 pixels, plus the coarse residual. The initial mask uses σ = 8, giving one further nominal mask width of 64.5 pixels.

Identical inputs and selections within each example.

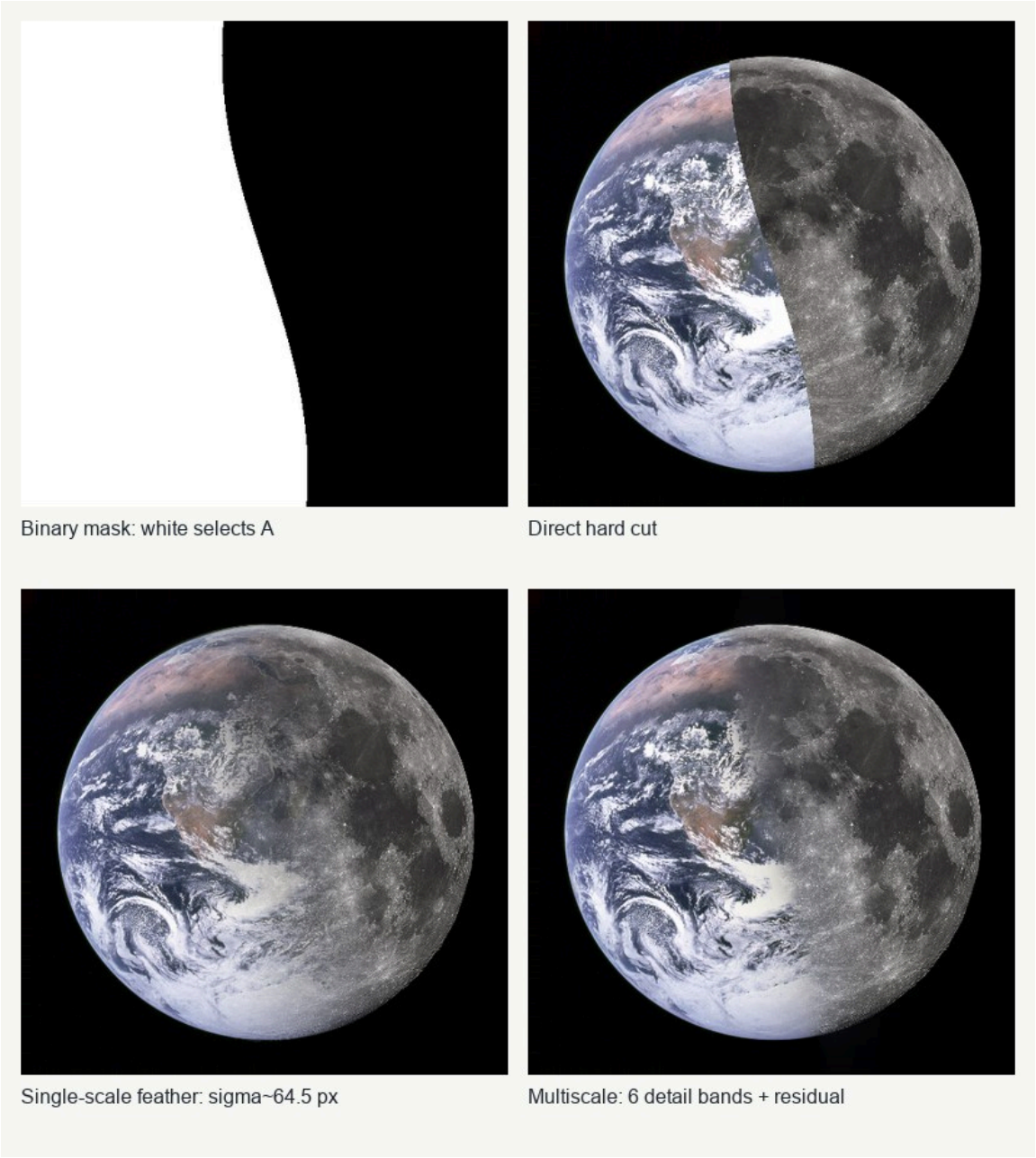
Example	Hard J	Feather J	Multi J	Hard Cε (%)	Feather Cε (%)	Multi Cε (%)
Oraple	0.1716	0.0122	0.0279	0.00	0.00	0.94
Sandstone arch and alpine lake	0.2752	0.1572	0.1669	0.00	0.00	0.13
Earth-Moon	0.2718	0.0422	0.0424	0.00	0.00	4.45

J is mean absolute RGB difference across adjacent pixels straddling the original binary contour, before clipping. Cε is the fraction of scalar values below -10^{-6} or above $1 + 10^{-6}$. This tolerance discounts roundoff and negligible excursions; strict counts and ranges remain in the measurements.

Feathering gives the smallest J but mixes more texture. A smaller boundary jump alone does not establish a better composite. The disk fit checks outline coincidence, not surface-feature correspondence or physical realism.

2.4 Planet blending: brightness and detail

The Earth–Moon blend uses a maximum image σ of 64 pixels because the brightness mismatch extends across much of the disk. Fewer bands leave a more visible boundary.



Three methods with the same registered inputs and binary selection. Initial mask smoothing at σ = 8 pixels reduces the abrupt contrast change where the selection meets the disk outline.

Multiresolution blending reduces the brightness step and keeps cloud and crater detail more localized than broad feathering. A binary finest mask leaves an abrupt outer-edge join; wider masks combine texture across a larger region. The same six initial widths and four-, five- and six-band comparisons are retained in the supplementary figures.

The composite remains an imaginary surface: lunar features do not continue Earth's clouds. Smoothing a boundary cannot reconcile different surface structures.

2.4 Color separates the source regions

The final composites use RGB. Orange sandstone, golden snowy peaks and blue water make the arch–lake source regions distinct. Blue oceans distinguish Earth from the nearly neutral Moon. These differences are less evident in grayscale, although the boundaries remain smoothed.



Sandstone arch and alpine lake / luminance



Sandstone arch and alpine lake / RGB



Earth-Moon blending / luminance



Earth-Moon blending / RGB

Luminance and RGB results with the same registration, masks and stack settings. Luminance is 0.2126R + 0.7152G + 0.0722B; RGB channels are blended independently.

Framing or registration, masks and stack settings are identical in this comparison. RGB channels are blended independently. Supplementary figures retain matched boundary crops, complete mask stacks, mask-width sweeps and band-count trials for both examples.

Additional parameter trials and diagnostic figures are available in the online supplement.

IMPLEMENTATION APPENDIX

A Convolution implementation

The shared helper validates the inputs, reverses the kernel, and pads the image with zeros. Both implementations return the same image shape and use the same padding convention.

Input validation, kernel reversal, and padding

```
def _convolution_inputs(image, kernel):
    image = np.asarray(image, dtype=np.float64)
    kernel = np.asarray(kernel, dtype=np.float64)
    if image.ndim != 2 or kernel.ndim != 2:
        raise ValueError('image and kernel must both be two dimensional')
    if not image.size or not kernel.size:
        raise ValueError('image and kernel must be nonempty')
    kh, kw = kernel.shape
    padded = np.pad(image, ((kh // 2, (kh - 1) // 2), (kw // 2, (kw - 1) // 2)),
mode='constant')
    return (image, kernel[::-1, ::-1], padded)
```

Four-loop convolution

```
def convolve_four_loops(image, kernel):
    image, flipped, padded = _convolution_inputs(image, kernel)
    height, width = image.shape
    kh, kw = flipped.shape
    out = np.zeros_like(image)
    for y in range(height):
        for x in range(width):
            for ky in range(kh):
                for kx in range(kw):
                    out[y, x] += flipped[ky, kx] * padded[y + ky, x + kx]
    return out
```

IMPLEMENTATION APPENDIX

B Vectorization and image gradients

The two-loop version accumulates a complete shifted image slice for each kernel element. This changes the Python loop structure without changing the convolution sum.

Two-loop convolution

```
def convolve_two_loops(image, kernel):
    image, flipped, padded = _convolution_inputs(image, kernel)
    height, width = image.shape
    kh, kw = flipped.shape
    out = np.zeros_like(image)
    for ky in range(kh):
        for kx in range(kw):
            out += flipped[ky, kx] * padded[ky:ky + height, kx:kx + width]
    return out
```

Gradient and edge magnitude

```
gx = convolve_two_loops(image, DX)
gy = convolve_two_loops(image, DY)
magnitude = np.sqrt(gx * gx + gy * gy)
edges = magnitude > threshold
```

Cameraman input preparation

The supplied image has a thin white border. The crop (1, 7, 535, 541), in (left, top, right, bottom) coordinates with the right and bottom bounds excluded, removes it. The square crop is resized to 256 × 256 pixels with Lanczos interpolation.

Calculations use floating-point intensities in [0, 1] with no further normalization before thresholding. The seven-pixel evaluation margin covers the largest Gaussian-derivative support used in Part 1.

IMPLEMENTATION APPENDIX

C Orientation without angle routines

For nonzero gradients, the ratio below lies in [0, 1]. When it exceeds $\sqrt{2}-1$, range reduction bounds the series argument by $\sqrt{2}-1$. Eighteen terms approximate arctangent; component swaps and signs recover the quadrant.

Range reduction

$$r = \min(|g_y|, |g_x|) \,/\, \max(|g_y|, |g_x|)$$

$$\operatorname{atan}(r) = \pi/4 + \operatorname{atan}((r-1)/(r+1)) \quad \text{for } r > \sqrt{2}-1$$

$$\operatorname{atan}(z) \approx z - z^3/3 + z^5/5 - \dots$$

Arithmetic-only orientation implementation

```
def orientation_without_angles(gx, gy, terms=18):
    gx, gy = np.broadcast_arrays(np.asarray(gx, float), np.asarray(gy, float))
    if not np.all(np.isfinite(gx)) or not np.all(np.isfinite(gy)):
        raise ValueError('gradient components must be finite')
    if terms < 1:
        raise ValueError('terms must be positive')
    ax, ay = (np.abs(gx), np.abs(gy))
    high, low = (np.maximum(ax, ay), np.minimum(ax, ay))
    ratio = np.divide(low, high, out=np.zeros_like(high), where=high != 0)
    reduce = ratio > np.sqrt(2.0) - 1.0
    z = np.where(reduce, (ratio - 1.0) / (ratio + 1.0), ratio)
    z2 = z * z
    poly = np.zeros_like(z)
    for n in range(terms - 1, -1, -1):
        poly = (-1.0) ** n / (2 * n + 1) + z2 * poly
    small_angle = z * poly + np.where(reduce, np.pi / 4, 0.0)
    angle = np.where(ay > ax, np.pi / 2 - small_angle, small_angle)
    angle = np.where(gx < 0, np.pi - angle, angle)
    angle = np.where(gy < 0, -angle, angle)
    return np.where(high == 0, 0.0, angle)
```

A zero gradient is assigned orientation zero and displayed black. NumPy atan2 is used only as an independent validation reference.

D Gaussian derivatives and sharpening

Full convolution retains the complete Gaussian-derivative support. The sharpening kernel combines the identity and Gaussian terms into one image convolution.

Build Gaussian derivative kernels

```
g1 = cv2.getGaussianKernel(13, 2.0)
G = g1 @ g1.T
Kx = convolve2d(G, DX, mode='full')
Ky = convolve2d(G, DY, mode='full')
gx = convolve2d(image, Kx, mode='same', boundary='fill')
gy = convolve2d(image, Ky, mode='same', boundary='fill')
```

Single-convolution sharpening kernel

```
def unsharp_kernel(gaussian, alpha):
    gaussian = np.asarray(gaussian, float)
    if gaussian.ndim != 2 or any((n % 2 == 0 for n in gaussian.shape)):
        raise ValueError('Gaussian must have odd dimensions')
    if not np.isclose(gaussian.sum(), 1.0):
        raise ValueError('Gaussian must sum to one')
    kernel = -float(alpha) * gaussian.copy()
    kernel[gaussian.shape[0] // 2, gaussian.shape[1] // 2] += 1 + float(alpha)
    return kernel
```

The sharpening kernel is convolved with each RGB channel using “same” output and symmetric boundary extension. All intermediate arrays use float64; clipping is deferred until display export.

E Hybrids and multiresolution stacks

The hybrid combines a low-pass source with a scaled high-pass source. The stack construction keeps every level at the original resolution; the last Gaussian level is retained as the coarse residual.

Hybrid synthesis

```
low = gaussian(A, sigma_low)
high = B − gaussian(B, sigma_high)
hybrid = low + gain ∗ high
```

Stack construction and reconstruction

```
G = [I] + [gaussian(I, 2 ∗ 2 ∗∗ i) for i in range(5)]
L = [G[i] − G[i + 1] for i in range(5)] + [G[−1]]
S = [M[i] ∗ LA[i] + (1 − M[i]) ∗ LB[i] for i in range(6)]
result = sum(S)
```

Numerical representation

Hybrid filtering uses reflected boundaries and signed float64 values. Intermediate bands are neither clipped nor contrast-normalized. Display exports clamp to [0, 1] and quantize to 8 bits; the settings table counts clipped scalar channel values. The signed high-pass display is shifted by 0.5 and includes the synthesis gain, without additional display amplification.

These excerpts show the filtering and reconstruction steps. The source ZIP contains the complete alignment, masking, validation, and figure-generation code.

SUPPORTING INFORMATION

R Reproduction and observations

Observations

Spatial scale determines which structures are preserved. Smoothing suppresses fine variation before differentiation; hybrid images separate broad shape from fine detail; multiresolution blending uses broader transitions for brightness and color than for texture. Numerical agreement checks confirm the implementations, but alignment and source geometry still limit the visual results.

Reproduce the experiments

The source ZIP includes the original inputs, parameters, implementations, tests, measurements, and report builder. The following commands regenerate the experiments and webpage. README.md describes PDF export and image replacement.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
python run_filters.py
python run_frequencies.py
python build_report.py
python -m http.server 8000 --directory site
```

Submit the report PDF and code + inputs ZIP together, and include the website URL. Measurements include the comparison diagnostics and level trace; the image manifest records source URLs, rights, dimensions and hashes.

References

1. A. Oliva, A. Torralba, and P. G. Schyns. Hybrid images. ACM Transactions on Graphics 25(3), 527–532, 2006.
2. P. J. Burt and E. H. Adelson. A multiresolution spline with application to image mosaics. ACM Transactions on Graphics 2(4), 217–236, 1983.
3. R. Szeliski. Computer Vision: Algorithms and Applications, second edition. Figure 3.42.
4. SYDE 671 Assignment 2, “Fun with Filters and Frequencies.” The required experiments follow the supplied handout, adapted from Berkeley CS180.
5. A. Oliva. The Art of Hybrid Images: Two for the View of One. Art & Perception 1, 65–74, 2013. Classic Monroe–Einstein example.

S Image credits

The self portrait and street photograph are the supplied Assignment 1 images. The Taj Mahal, Derek/Nutmeg pair, and original apple and orange PNGs are supplied course samples. The custom examples use the credited source photographs.

Image	Credit	Rights / provenance	Link
Derek portrait	Derek Hoiem / course materials	Course sample; see note below.	Source
Nutmeg the cat	Derek Hoiem / course materials	Course sample; see note below.	Source
Taj Mahal	Course materials; individual photographer not specified	Course sample; see note below.	Source
Classic cameraman	Course materials; individual photographer not specified	Course sample; see note below.	Source
Apple (supplied Burt-Adelson original)	Peter J. Burt and Edward H. Adelson / course materials	Course sample; see note below.	Supplied Part 2 image package
Orange (supplied Burt-Adelson original)	Peter J. Burt and Edward H. Adelson / course materials	Course sample; see note below.	Supplied Part 2 image package
Moksh portrait	User-supplied photograph	User-provided photograph.	Supplied photograph
Street photograph	User-supplied photograph	User-provided photograph.	Supplied photograph
The Blue Marble / Earth seen from Apollo 17	NASA / Apollo 17 crew	Public domain (U.S. government work)	Source License

Course samples are supplied for this assignment; no general open license is asserted. Personal photographs are user-provided.

S Image credits, continued

The following source records complete the input-image manifest. Adapted figures retain the applicable attribution and license information.

Image	Credit	Rights / provenance	Link
Full Moon, 22 October 2010	Gregory H. Revera	CC BY-SA 3.0	Source License
Barack Obama official portrait, December 2012	Pete Souza / White House	Public domain (U.S. government work)	Source
Albert Einstein portrait (1947)	Orren Jack Turner; Library of Congress	Public domain in the United States (copyright not renewed); Library of Congress reports no known restrictions on publication.	Source License
Joe Biden official presidential portrait, 2021	Adam Schultz / White House	Public domain (U.S. government work)	Source License
Turret Arch through North Window (crop)	Thomas Wolf, www.foto-tw.de	CC BY-SA 3.0	Source License
Alpenglow on Lake McDonald	David Restivo / NPS (GlacierNPS)	CC BY 2.0	Source License
Marilyn Monroe in Niagara (1953 publicity portrait)	Gene Kornman / 20th Century Fox	Public domain in the United States (copyright not renewed).	Source License

Adapted figures containing the Moon photograph by Gregory H. Revera remain under CC BY-SA 3.0. Complete attribution and modification records accompany the code and image manifest.